Instituto Politécnico Nacional

Escuela Superior de Cómputo

Procesamieno Digital de Imágenes

Segmentación facial para la identificación de menores desaparecidos

INTEGRANTES:

Domínguez Salas Ernesto

Gutiérrez Sánchez Angélica

Montero Molina Alberto

Reyes Martínez Ulises

Valdez García Paola

COMUNIDAD 3

Grupo 4BM1

María Elena Cruz Meza

16 de diciembre de 2024

Índice

1. Introducción **3**
1.1. Descripción del problema . . . 3
1.2. Objetivos . . . 4
1.2.1. Objetivo general . . . 4
1.2.2. Objetivos específicos . . . 4
1.3. Organización del documento . . . 4

2. Conceptos básicos **6**
2.1. Imagen . . . 6
2.2. Histograma . . . 7

3. Marco teórico **10**
3.1. Técnicas de manipulación del histograma . . . 10
3.1.1. Desplazamiento . . . 10
3.1.2. Contracción . . . 10
3.1.3. Expansión . . . 10
3.2. Operaciones puntuales . . . 10
3.2.1. Conversión a escala de grises . . . 10
3.2.2. Binarización . . . 11
3.2.3. Operaciones aritméticas . . . 11
3.2.4. Operaciones lógicas . . . 12
3.3. Fitrado . . . 13
3.3.1. Filtros lineales . . . 13
3.3.2. Filtros no lineales . . . 14

4. Segmentación de rostros en imágenes digitales **16**
4.1. Herramientas de desarrollo . . . 16
4.2. Conjunto de imágenes . . . 16
4.3. Ajuste de brillo . . . 17
4.3.1. Ecualización exponencial . . . 17
4.4. Eliminación de ruido . . . 20
4.4.1. Ruido gaussiano . . . 20
4.4.2. Ruido sal y pimienta . . . 22
4.4.3. Filtro gaussiano . . . 24
4.4.4. Filtro moda . . . 27
4.5. Segmentación parcial y total . . . 30
4.5.1. Filtro Roberts . . . 30
4.5.2. Umbral banda . . . 33

5. Resultados **35**
5.1. Pruebas en imágenes obscuras . . . 35
5.2. Pruebas en imágenes brillantes . . . 35
5.3. Pruebas en imágenes con alto contraste . . . 36
5.4. Pruebas en imágenes con bajo contraste . . . 36

6. Conclusiones **37**

Anexos **38**

Referencias **52**

1. Introducción

En este documento se exploran y desarrollan técnicas destinadas a abordar problemas como el ajuste del brillo y la reducción de ruido, factores que impactan significativamente la precisión en la detección de rostros. Asimismo, se implementan estrategias de segmentación parcial y total, con el propósito de mejorar el análisis y la extracción de características faciales, contribuyendo a la identificación de menores desaparecidos.

1.1. Descripción del problema

En México, la privación de la libertad de menores es alarmante y ha mostrado un incremento preocupante en los últimos años. Según datos del Secretariado Ejecutivo del Sistema Nacional de Seguridad Pública (SESNSP), en 2023 se reportaron más de 5,000 menores desaparecidos en el país, lo que equivale a un promedio de 14 desapariciones diarias. Gran parte de los casos involucra a adolescentes de entre 12 y 17 años, siendo las niñas y adolescentes las más afectadas, representando más del 60 por ciento de las desapariciones (SESNSP, 2024).

Los estados con las cifras más altas de secuestros de menores incluyen la Ciudad de México, Veracruz y el Estado de México, que concentran un alto número de casos y enfrentan desafíos críticos en la lucha contra la delincuencia organizada y el crimen violento (Figura 1).

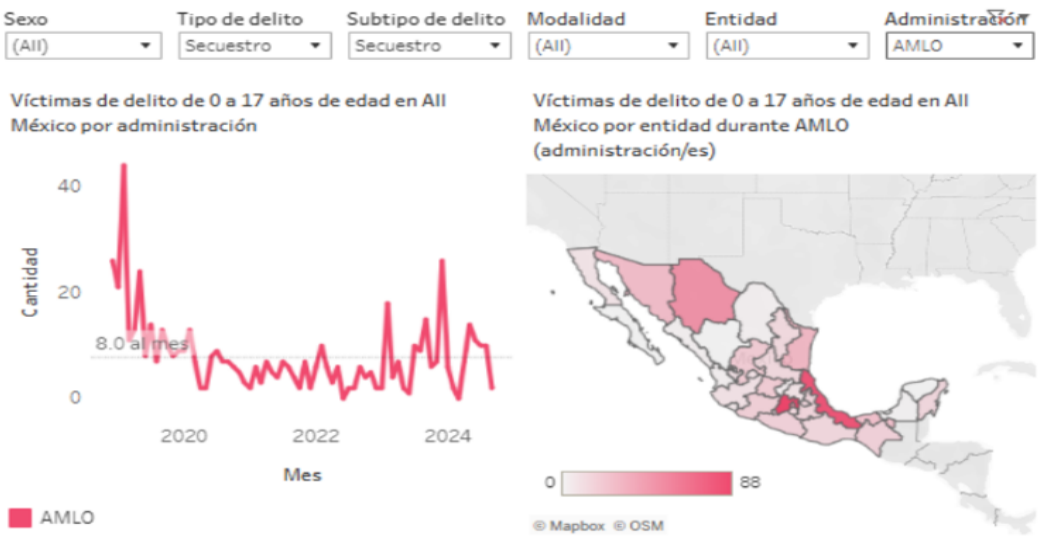


Figura 1: Víctimas de secuestro de 0 a 17 años en México de 2020 a 2024. Secuestro de niñas, niños y adolescentes en México (a septiembre de 2024). Obtenido de https://blog.derechosinfancia.org.mx

En el contexto de la búsqueda de menores, el Procesamiento Digital de Imágenes puede contribuir significativamente a agilizar los procesos de localización y optimizar la identificación de personas en imágenes digtales obtenidas de cámaras de seguridad o redes sociales. Sin embargo, la segmentación de rostros presenta varios desafíos significativos que deben ser abordados para mejorar la precisión y eficacia de los sistemas de reconocimiento facial.

Uno de los principales problemas radica en las condiciones de iluminación en las que se capturan las imágenes, es decir, pueden estar demasiado oscuras o, por el contrario, excesivamente brillantes (Figura 2), lo que dificulta la identificación de características faciales clave. Estas variaciones en la iluminación

pueden llevar a que ciertas partes del rostro queden ocultas o distorsionadas, afectando así el rendimiento de los algoritmos de segmentación.

Por otro lado, la presencia de ruido en las imágenes, ya sea de tipo sal y pimienta o ruido gaussiano, puede introducir distorsiones que complican aún más la tarea de segmentación. Estos ruidos pueden enmascarar detalles importantes de los rasgos faciales, lo que resulta en una identificación incorrecta o incompleta.

1.2. Objetivos

1.2.1. Objetivo general

Crear una aplicación para la segmentación facial en imágenes digitales que presentan problemas de iluminación, ruido sal y pimienta o gaussiano, variabilidad de texturas, tonos de piel, así como el traslape entre diferentes objetos.

1.2.2. Objetivos específicos

- Investigación de algunas técnicas de procesamiento digital de imágenes para ajuste de brillo, reducción de ruido y detección de bordes.
- Análisis e implementación de la técnica de manipulación del histograma "Ecualización exponencial" para mejorar el brillo de una imagen.
- Análisis e implementación de las técnicas de eliminación de ruido "Filtro Gaussiano" y "Filtro Moda", para homogeneizar las regiones u objetos de interés.
- Análisis e implementación de las técnicas de extracción de bordes, para segmentar parcialmente y completamente la región de interés.
- Integración de los prototipos para la segmentación total de rostros.

1.3. Organización del documento

El documento se organiza de la siguiente manera: En el capítulo 1 se incluye la descripción del problema, así como los objetivos general y específicos con el propósito de contextualizar la problemática de la desaparición de menores y justificar la necesidad de desarrollar herramientas tecnológicas para su localización.

En el capítulo 2 se presentan los conceptos fundamentales del procesamiento digital de imágenes, necesarios para comprender las técnicas implementadas en este trabajo. Estos conceptos incluyen la definición y clasificación de imágenes, así como el uso del histograma para analizar sus propiedades visuales.

En el Capítulo 3 se abordan las bases teóricas que sustentan el trabajo, como las técnicas de manipulación del histograma (desplazamiento, contracción y expansión), esenciales para ajustar el brillo y el contraste. También se abordan operaciones puntuales (conversión a escala de grises, binarización y operaciones aritméticas y lógicas) y técnicas de filtrado lineal y no lineal, fundamentales para la mejora y segmentación de imágenes.

En el capítulo 4 se describe la solución desarrollada para mejorar la calidad de las imágenes y facilitar la segmentación facial en condiciones adversas. Se incluyen técnicas como la ecualización exponencial para el ajuste de brillo, métodos de filtrado (filtro gaussiano y filtro moda) para eliminar ruido, y enfoques de segmentación (filtro de Roberts y umbral banda) para resaltar áreas de interés.

En el capítulo 5 se muestran los resultados obtenidos tras aplicar las técnicas desarrolladas a diferentes tipos de imágenes: obscuras, brillantes, con alto contraste y con bajo contraste. Cada caso muestra las etapas clave del proceso, que incluyen el ajuste de brillo, filtrado, conversión a escala de grises, segmentación parcial y total, y la validación de bordes mediante operaciones lógicas.

Finalmente en el capítulo 6 se presentan las conclusiones derivadas del trabajo realizado, detallando la implementación de técnicas de ajuste de brillo, eliminación de ruido y detección de bordes, destacando el cumplimiento del objetivo general del proyecto apartir de las tareas realizadas y las pruebas efectuadas.

2. Conceptos básicos

2.1. Imagen

Una imagen es una representación visual de un objeto, escena o concepto capturada mediante un medio físico o digital. La imagen se representa, desde el punto de vista computacional, como una matriz numérica $f(x, y)$ de dimensión $M \times N$, es decir, con M filas y N columnas. El contenido de esta matriz se refiere a los valores escalares que se cuantifican en un rango entre 0 y 255, y que además están situados en localizaciones espaciales (x, y) denominados píxeles como se muestra en la Figura 2.

Sobre cada matriz se establece un sistema de coordenadas con origen normalmente en la esquina superior izquierda y con los ejes x e y tales que su orientación es positiva hacia al derecha en el caso del eje x y hacia abajo en el caso del eje y.

$f(x, y) =$

75	78	83	91	91	94	103	110	113	121
75	77	80	89	91	93	103	110	115	123
74	77	79	88	91	93	103	111	118	126
74	77	80	86	91	94	103	114	121	130
74	77	81	86	90	97	107	117	127	135
75	78	82	86	90	101	116	123	132	140
75	80	85	88	94	109	120	129	139	145
75	80	86	89	98	112	124	137	146	153
75	81	85	93	106	118	132	144	152	160
75	82	87	97	112	122	135	148	157	162

Figura 2: Representación de una imagen digital en escala de grises

Existen diferentes tipos de imágenes, las que están en color, en escala de grises y en binario como se muestra en la Figura 3. Las primeras contienen información sobre la intensidad de los colores en cada píxel, generalmente se representan en el espacio de color RGB (Rojo, Verde, Azul). En este modelo, cada píxel está compuesto por tres valores: intensidad del componente rojo, azul y verde; cada uno de estos puede tener un valor entre 0 y 255, permitiendo así más de 16 millones de combinaciones de colores.

A diferencia de las imagenes en color, las imágenes en escala de grises contienen solo información sobre la intensidad de luz. Cada píxel en la imagen tiene un único valor que representa su brillo, sin información de color. En un sistema de 8 bits, cada píxel puede tener un valor entre 0 (negro) y 255 (blanco), con 256 niveles de gris posibles. Esto permite representar diferentes tonos de gris.

Por otro lado, las imágenes binarias son una forma extrema de imágenes en escala de grises, donde cada píxel puede tener solo uno de dos valores: 0 (negro) o 1 (blanco). Esto las hace ideales para representar datos en un formato simplificado. Es importante mencionar que son útiles para identificar objetos dentro de una imagen y separar el fondo del primer plano.

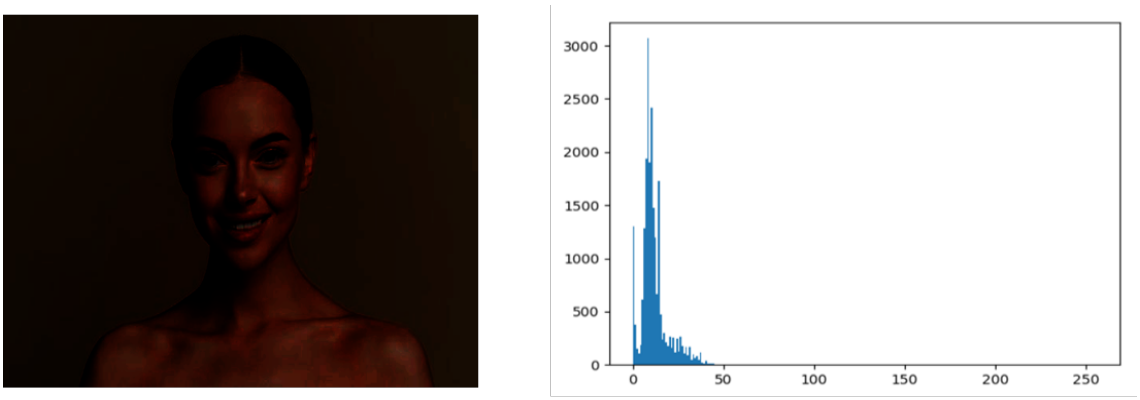
a. b. c.

Figura 3: a) Imagen hombre color. b) Imagen hombre escala de grises. c) Imagen hombre binaria

2.2. Histograma

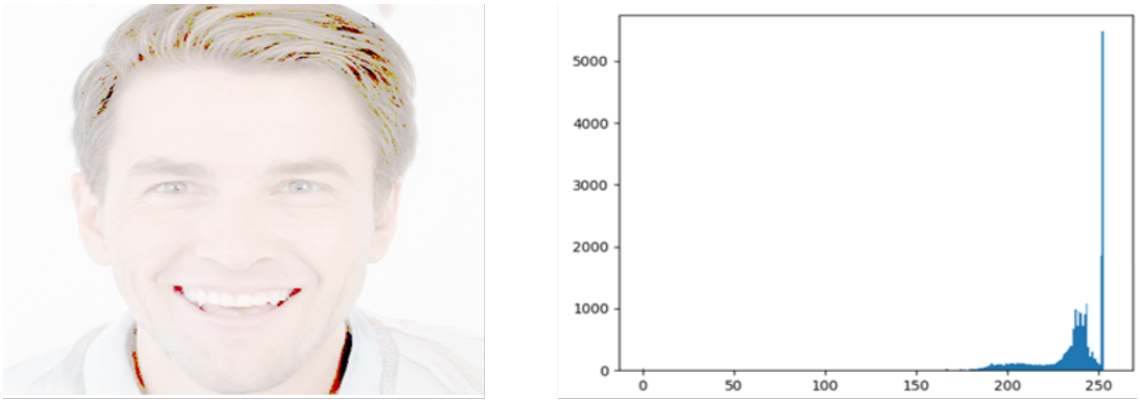
El histograma de una imagen es una representación gráfica de la distribución de los niveles de intensidad de los píxeles en una imagen digital (González & Woods, 2018). En una imagen en escala de grises de 8 bits, el histograma abarca valores de intensidad de 0 (negro) a 255 (blanco), mostrando la cantidad de píxeles que corresponden a cada valor específico (Castleman, 1996). Esto permite analizar características de brillo y contraste, así como la distribución de tonos en la imagen.

Si la mayoría de los valores de intensidad se concentra en la parte izquierda del histograma, la imagen tiende a ser oscura, mientras que, si los valores están hacia la parte derecha, la imagen es más brillante (González & Woods, 2018). Una distribución uniforme a lo largo del histograma indica un alto contraste, lo que sugiere que la imagen tiene un rango completo de detalles visibles en las áreas de sombras y luces. Finalmente, si el histograma está distribuido a lo largo del eje x y no rebasa el valor de 180 indica bajo contraste en la imagen.



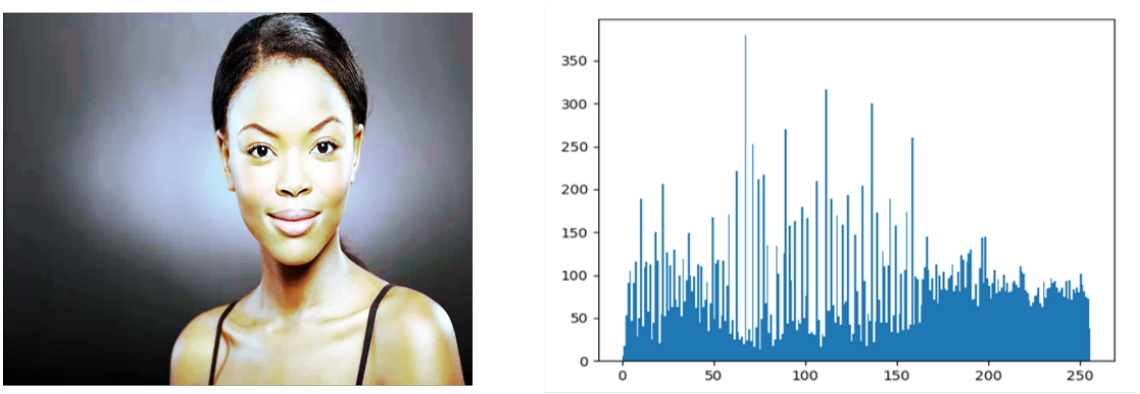
a. b.

Figura 4: a) Imagen mujer oscura. b) Histograma de imagen mujer oscura.



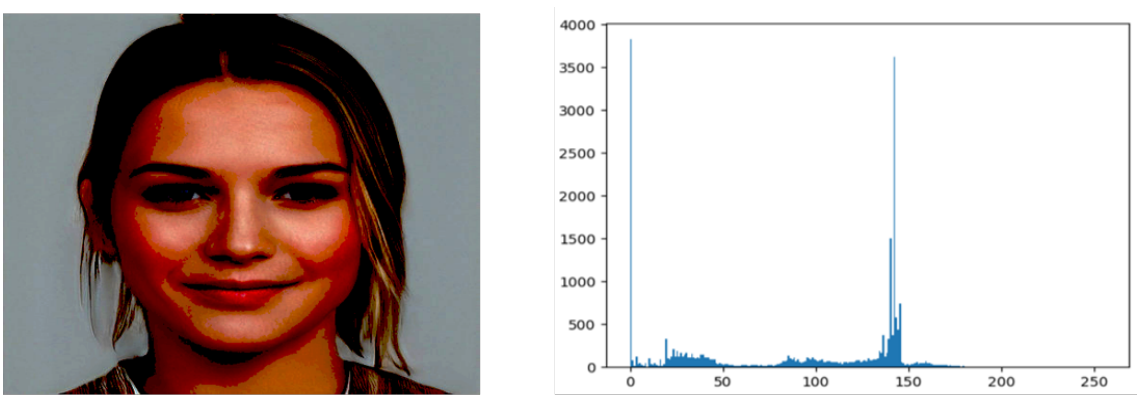
a. b.

Figura 5: a) Imagen hombre brillante. b) Histograma de imagen hombre brillante.



a. b.

Figura 6: a) Imagen mujer alto contraste. b) Histograma de imagen mujer alto contraste.



a. b.

Figura 7: a) Imagen mujer bajo contraste. b) Histograma de imagen mujer bajo contraste.

El análisis del histograma permite aplicar técnicas de mejora de brillo y contraste conocidas como técnicas de manipulación del histograma; entre ellas se encuentran el desplazamiento, la expansión y la contracción del histograma, dichas técnicas ajustan el rango de intensidades de la imagen.

3. Marco teórico

3.1. Técnicas de manipulación del histograma

3.1.1. Desplazamiento

Según González & Woods (2018), el desplazamiento es el cambio uniforme de todos los niveles de intensidad hacia valores más altos o más bajos en el histograma.Si los valores se desplazan hacia la derecha (valores más altos), la imagen se aclara (aumenta el brillo); por el contrario si se desplazan hacia la izquierda (valores más bajos), la imagen se oscurece (reduce el brillo).

Si $I(x, y)$ es la intensidad original en la posición (x, y) y c es una constante:

$$I'(x, y) = I(x, y) + c$$

- Si $c > 0$: Desplazamiento hacia valores más altos (la imagen se aclara).
- Si $c < 0$: Desplazamiento hacia valores más bajos (la imagen se oscurece).

3.1.2. Contracción

Es la reducción del rango de niveles de intensidad. Los valores del histograma se comprimen hacia el centro del rango dinámico ocasionando que la imagen pierda contraste, ya que los tonos oscuros se aclaran y los tonos claros se oscurecen (Jain, 1989).

$$I_{\text{out}}(x, y) = \frac{I_{\text{in}}(x, y) - I_{\text{min}}}{I_{\text{max}} - I_{\text{min}}} \times (L_{\text{max}} - L_{\text{min}}) + L_{\text{min}}$$

3.1.3. Expansión

Es el aumento del rango de niveles de intensidad. Los valores del histograma se estiran hacia los extremos (0 y 255 en una imagen de 8 bits) mejorando el contraste al hacer que las áreas claras sean más claras y las oscuras más oscuras así como resaltando detalles que podrían no ser visibles en una imagen con bajo contraste (Jain, 1989).

$$I_{\text{out}}(x, y) = a + \frac{I(x, y) - I_{\text{min}}}{I_{\text{max}} - I_{\text{min}}} \times (b - a)$$

3.2. Operaciones puntuales

3.2.1. Conversión a escala de grises

Según Gonzalez & Woods (2018), la conversión de una imagen en color a escala de grises es una técnica de procesamiento de imágenes que reduce una imagen a diferentes niveles de intensidad de gris, eliminando la información de color. Este proceso facilita el análisis de imágenes en términos de brillo y contraste sin distracciones causadas por la cromaticidad.

Existen varios métodos para realizar esta conversión, cada uno con sus ventajas y aplicaciones según el contexto de uso (Jain, 1989). La media simple es un método que calcula el promedio de las intensidades de los tres canales (rojo, verde y azul) para cada píxel, la formula que representa este método es la siguiente:

$$I_{\text{out}}(x, y) = \frac{R(x, y) + G(x, y) + B(x, y)}{3}$$

Según Jain (1989) también existe el método ponderado, el cual tiene en cuenta cómo el ojo humano percibe los diferentes colores, ya que no todos los colores tienen la misma importancia en términos de brillo. Los coeficientes 0.299, 0.587 y 0.114 son valores que están ajustados para reflejar la sensibilidad del ojo a los diferentes colores

Como señala Russ (2016), el verde tiene la mayor contribución porque el ojo humano es más sensible a la luz verde, lo que explica por qué tiene un coeficiente más alto, mientras que el rojo y el azul tienen coeficientes más bajos, con el rojo teniendo un coeficiente ligeramente más alto debido a su contribución relativamente mayor al brillo percibido en comparación con el azul.

$$I_{\text{out}}(x, y) = 0{,}299 \cdot R(x, y) + 0{,}587 \cdot G(x, y) + 0{,}114 \cdot B(x, y)$$

3.2.2. Binarización

Es el proceso de convertir una imagen a formato binario (valores de píxeles 0 o 255), dependiendo de un umbral. Este proceso simplifica la imagen para destacar áreas de interés, como bordes o regiones con alta variación de intensidad, siendo muy útil en tareas de segmentación y reconocimiento de objetos (Gonzalez & Woods 2018).

$$I_{\text{out}}(x, y) = \begin{cases} 255, & \text{si } I(x, y) > T \\ 0, & \text{si } I(x, y) \leq T \end{cases}$$

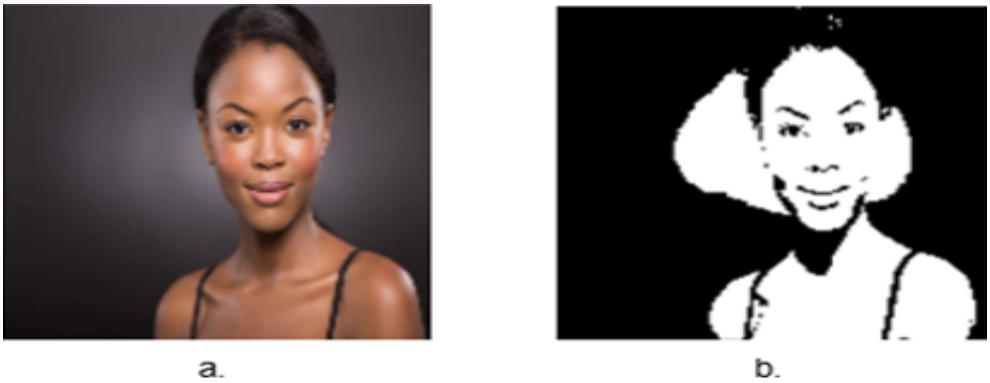


Figura 8: a) Imagen mujer morena color . b) Imagen binaria de la imagen mujer morena color

3.2.3. Operaciones aritméticas

Según Castleman (1996), las operaciones aritméticas en procesamiento de imágenes son técnicas de procesamiento que combinan dos o más imágenes a nivel de píxel mediante operaciones matemáticas básicas. Estas operaciones permiten mezclar, resaltar o mejorar ciertas características en las imágenes y son ampliamente utilizadas en aplicaciones como la mejora de contraste, eliminación de ruido, fusión de imágenes y análisis de movimiento.

La suma aritmética entre dos imágenes consiste en combinar las intensidades de los píxeles correspondientes de ambas imágenes de entrada para generar una nueva imagen como resultado. Este proceso permite fusionar la información contenida en las imágenes originales, destacando especialmente las regiones brillantes comunes, lo que puede ser útil para resaltar detalles compartidos o para crear efectos visuales interesantes (Castleman, 1996).

Si $I_1(x, y)$ y $I_2(x, y)$ son las intensidades de los píxeles en la posición (x, y) de las dos imágenes de entrada, la intensidad del píxel en la imagen resultante:

$$I_{\text{out}}(x, y) = I_1(x, y) + I_2(x, y)$$

a. b. c.

Figura 9: a) Imagen mujer morena color . b) Imagen hombre color. c) Imagen resultante de la suma entre imagen mujer morena color e imagen hombre color.

Existen otras operaciones aritméticas que pueden realizarse entre dos imágenes, como son la resta y la multiplicación, la primera es útil para resaltar las diferencias entre las imágenes, especialmente cuando han sido capturadas en tiempos o condiciones diferentes; además es importante mencionar que puede producir valores negativos, por lo que es común usar valores absolutos o reajustar los niveles de intensidad para evitar resultados negativos. En cambio, la multiplicación es útil para realizar enmascaramiento o atenuación de una imagen; las regiones donde uno de los píxeles es 0 en cualquiera de las imágenes darán como resultado un píxel con valor 0 (Castleman, 1996).

Es importante mencionar que lasoperaciones aritméticas entre dos imágenes puede dar lugar a valores de intensidad que exceden el rango permitido (por ejemplo, más de 255 en imágenes de 8 bits), por lo que es necesario un ajuste (clipping) para mantener los valores dentro del rango.

3.2.4. Operaciones lógicas

Son transformaciones basadas en operaciones lógicas básicas aplicadas a nivel de píxel. Estas operaciones se usan principalmente para combinar imágenes en función de condiciones específicas, crear máscaras, y resaltar o suprimir regiones en una imagen en función de otra. Las operaciones lógicas son fundamentales en el procesamiento de imágenes binarias, pero también pueden aplicarse en imágenes en escala de grises o en color para ciertas aplicaciones (Jain, 1989).

Según Pratt (2007), la operación AND es una operación lógica muy utilizada, toma dos imágenes y compara cada par de píxeles correspondientes. Si ambos píxeles son "1"(en el caso de imágenes binarias, blanco), entonces el píxel resultante es "1"; en cualquier otro caso, el píxel resultante es "0"(negro).

Si $I_1(x, y)$ y $I_2(x, y)$ son las intensidades de los píxeles en la posición (x, y) de dos imágenes binarias, la operación lógica AND produce una nueva imagen donde:

$$I_{\text{out}}(x, y) = I_1(x, y) \wedge I_2(x, y)$$

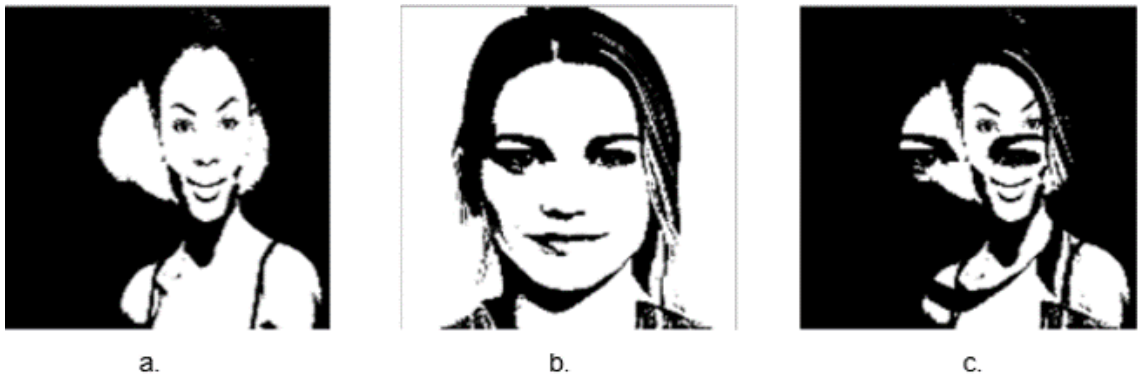


Figura 10: a) Imagen mujer morena binaria . b) Imagen mujer rubia binaria. c) Imagen resultante de la operación AND entre imagen mujer morena binaria e imagen mujer rubia binaria

Entre otras operaciones lógicas, se encuentra la OR y XOR. La primera compara cada par de píxeles correspondientes. Si al menos uno de los píxeles es "1", entonces el píxel resultante es "1"; solo es "0"si ambos píxeles son "0". Permite combinar dos o más máscaras para crear una máscara más amplia que abarque todas las regiones de interés.

Por otro lado, la operación XOR compara cada par de píxeles de dos imágenes y da como resultado "1"solo si uno de los píxeles es "1ƨ el otro es "0". Si ambos son iguales (ambos "1.º ambos "0"), el resultado es "0".

3.3. Fitrado

El filtrado es una técnica fundamental en PDI que se utiliza para mejorar la calidad de la imagen eliminando o reduciendo el ruido y resaltando características específicas. En el contexto de imágenes digitales, el ruido puede introducirse por diversas fuentes, como condiciones de baja iluminación, interferencias electrónicas, compresión o durante la transmisión (Pratt, 2007).

Para eliminar o reducir este ruido, se utilizan varios tipos de filtros y técnicas de eliminación de ruido, que se seleccionan en función del tipo de ruido y de los detalles que se desean preservar en la imagen (González & Woods, 2018). Estos filtros se dividen en lineales y no lineales.

3.3.1. Filtros lineales

Un filtro lineal transforma una imagen a través de una combinación lineal de los valores de los píxeles vecinos. La operación más común en estos filtros es la convolución. Esta técnica implica el uso de un kernel (o máscara), que es una matriz de coeficientes para definir el tipo de filtro que se aplicará a la imagen (González & Woods, 2018).

La operación de convolución es la siguiente:

$$I_{\text{out}}(x, y) = \sum_{i=-k}^{k} \sum_{j=-l}^{l} I_{\text{in}}(x + i, y + j) \cdot K(i, j)$$

Los filtros lineales se clasifican principalmente en dos: filtros paso-bajas y filtros paso-altas. La función principal de los filtros paso-bajas es suavizar una imagen, eliminar ruido y reducir detalles finos, lo que resulta en una imagen más uniforme; permiten el paso de frecuencias bajas mientras atenúan o eliminan las frecuencias altas. Entre los más comunes se encuentran el filtro promedio, el cual hace uso de la media de los píxeles en un vecindario para reemplazar el valor de cada píxel de la imagen (Jain, 1989).

Matemáticamente el valor del píxel $I_{\text{out}}(x, y)$ en la imagen de salida se calcula como:

$$I_{\text{out}}(x, y) = \frac{1}{(2k+1)(2l+1)} \sum_{i=-k}^{k} \sum_{j=-l}^{l} I_{\text{in}}(x+i, y+j)$$

A diferencia de los filtros paso-bajas, los filtros paso-altas se utilizan para resaltar las frecuencias altas mientras atenúan o eliminan las frecuencias bajas. Este tipo de filtrado es esencial para la detección de bordes, el realce de detalles y la eliminación de componentes suaves en una imagen. Algunos de estos filtros son el filtro Sobel y filtro Laplaciano (González & Woods, 2018).

El filtro laplaciano se basa en la segunda derivada de la imagen, lo que significa que detecta los puntos donde la intensidad de la imagen cambia de manera más rápida (González & Woods, 2018).

Es relevante recordar que el Laplaciano es la suma de las segundas derivadas parciales de una función $f(x, y)$ representando la imagen en el espacio de las intensidades de los píxeles:

$$\Delta f(x, y) = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

El operador Laplaciano se aplica a una imagen a través de una máscara o kernel, que realiza una convolución con la imagen para calcular las segundas derivadas, normalmente se ocupa una máscara de 3x3. La máscara básica de este filtro es la siguiente:

$$\text{Máscara Laplaciana} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

Este enfoque ayuda a mejorar los resultados, especialmente en imágenes ruidosas.

3.3.2. Filtros no lineales

Estos filtros no aplican una combinación lineal de los valores de los píxeles vecinos. Los filtros no lineales modifican el valor de un píxel basándose en una función que toma en cuenta la relación entre el píxel central y sus vecinos, sin requerir que la salida sea proporcional a la entrada. Esto permite que los filtros no lineales sean más flexibles y eficaces en diversas aplicaciones, particularmente en situaciones donde se requiere un alto grado de adaptabilidad y resistencia al ruido. El filtro de mediana es un ejemplo de este tipo de filtros, es especialmente útil para eliminar ruidos de tipo "sal y pimienta", que son típicos en imágenes digitalizadas, donde algunos píxeles toman valores extremadamente altos o bajos en comparación con sus vecinos (Szeliski, 2010).

El filtro mediana no se basa en una máscara fija de valores numéricos como los filtros lineales (por ejemplo, el filtro de media). En cambio, se aplica sobre una vecindad de píxeles alrededor de cada píxel central, y el valor de salida es simplemente el valor mediano de esos píxeles. Sin embargo, a veces se puede hablar de la máscara en términos de la ventana de píxeles que se utiliza, por ejemplo 3x3 (Jain, 1989).

Para un filtro mediana de 3x3, se tiene la siguiente matriz:

$$\begin{bmatrix} I(x-1,y-1) & I(x-1,y) & I(x-1,y+1) \\ I(x,y-1) & I(x,y) & I(x,y+1) \\ I(x+1,y-1) & I(x+1,y) & I(x+1,y+1) \end{bmatrix}$$

En este caso, el valor de $I_(x,y)$ es reemplazado por la mediana de todos los valores dentro de la vecindad 3x3.

4. Segmentación de rostros en imágenes digitales

La segmentación de rostros es un proceso en el campo del procesamiento digital de imágenes y visión por computadora cuyo objetivo es identificar y aislar las regiones de una imagen que contienen un rostro humano. Este proceso es crucial en aplicaciones de reconocimiento facial (Zhao, Chellappa, & Rosenfeld, 2003).

A continuación se describen los algoritmos propuestos para segmentar rostros en imágenes digitales.

4.1. Herramientas de desarrollo

En el desarrollo del proyecto se emplearon herramientas clave que facilitaron tanto la implementación como la gestión de las operaciones necesarias. Windows (Versión 10.0.19045.5131) proporcionó un entorno estable y compatible para ejecutar los programas y bibliotecas requeridas, asegurando un flujo de trabajo eficiente.

Como editor de texto y entorno de desarrollo integrado, Visual Studio Code 2022 destacó por sus funcionalidades como el autocompletado, la depuración del código y su terminal integrada, lo que permitió una programación organizada y fluida.

Python (Versión 3.13) se utilizó como lenguaje de programación , pues su versatilidad y bibliotecas especializadas permitieron implementar algoritmos de PDI, diseñar la interfaz gráfica e integrar funciones para la segmentación.

Para la implementación de los algoritmos de procesamiento digital de imágenes se utilizaron las bibliotecas :

- OpenCV (Versión 4.10.0.84) proporciona funciones para manipular y analizar imágenes, como leer, escribir, convertir, y aplicar filtros y transformaciones a imágenes.
- NumPy (Versión 2.1.2) permite manejar y procesar tanto matrices como arreglos, lo que facilita la aplicación de operaciones matemáticas en imágenes.
- Tincker (Version 8.6) agiliza las opciones de diseño de interfaces gráficas para cargar imágenes, realizar operaciones y visualizar resultados.
- Pillow (Versión 11.0.0) ayuda con la conversión de imágenes procesadas por OpenCV en un formato compatible con Tkinter, para que estas puedan visualizarse correctamente en la interfaz gráfica.
- Matplotlib (Versión 3.9.2) contribuye a la creación de gráficos y visualizaciones permitiendo mostrar en este caso los histogramas de las imágenes.

4.2. Conjunto de imágenes

Se utilizó principalmente un conjunto de 4 imágenes en formato BMP para la experimentación de las técnicas de procesamiento digital de imágenes. Estas imágenes fueron seleccionadas estratégicamente para abarcar diferentes características y condiciones de iluminación: una con alto contraste (imagen 4-a), una con bajo contraste (imagen 5-a), una oscura (imagen 6-a) y una brillante (imagen 7-a).

Estas características permitieron evaluar la efectividad de las técnicas aplicadas, como ajustes de brillo y contraste, eliminación de ruido y detección de bordes, adaptándolas a las necesidades específicas de cada tipo de imagen.

4.3. Ajuste de brillo

Según González & Woods (2018), el ajuste de brillo es una operación fundamental en el PDI, utilizada para aumentar o disminuir la claridad general de una imagen. Al modificar el brillo, se cambia la intensidad de los píxeles, lo cual es útil para mejorar la visibilidad de los detalles en una imagen o adaptarla a ciertas condiciones de visualización.

4.3.1. Ecualización exponencial

La ecualización exponencial es una técnica que ajusta los valores de intensidad de una imagen mediante una transformación exponencial, lo que amplifica ciertos rangos de brillo más que otros. Esta técnica puede ser especialmente útil en situaciones donde se requiere un cambio más gradual en el brillo y contraste, con el objetivo de conservar detalles en áreas tanto oscuras como claras (Castleman, 1996).

La transformación en la ecualización exponencial se representa generalmente por:

$$I_{\text{exp}} = g_{\text{min}} - \frac{1}{\alpha} \ln(1 - \text{CDF}(I))$$

Donde

- $g_{\mathbf{min}}$ = valor mínimo de intensidad que puede tener la imagen después de la transformación.
- α = parámetro de control que ajusta la intensidad de la transformación exponencial.
- $\text{CDF}(I) =$ Función de distribución acumulativa (CDF) del nivel de intensidad I, la cual se calcula con la fórmula:

  $$\text{CDF}(I) = \frac{\sum_{j=0}^{I} H(j)}{\text{Total de píxeles}}$$

  donde $H(j)$ = Frecuencia de píxeles con intensidad j en la imagen (tomada del histograma).

A continuación se presenta un ejemplo de aplicación de ecualización exponencial.

Sea $I_(x, y)$ una imagen con dimensión de 10x10 con un rango de valores entre 0 y 255 (Figura 11).

142	142	141	142	142	140	144	45	0	2
143	143	142	142	140	146	108	1	1	0
143	142	142	142	140	147	47	0	2	0
142	143	143	143	144	122	6	0	0	0
143	142	143	141	149	77	0	2	0	0
143	143	143	141	147	40	0	2	0	0
142	143	142	143	133	13	0	1	0	0
142	143	142	147	112	0	0	0	0	6
143	143	141	149	96	0	2	0	1	23
144	143	140	150	75	0	3	0	7	34

Figura 11: Imagen con bajo contraste

El cálculo de la ecualización exponencial se aprecia en la Tabla 1 en donde se describe el nivel de gris, se calcula la frecuencia, la probabilidad, la distribución acumulativa y se muestra el valor final del pixel después de haber sido ecualizado. La imagen ecualizada completamente se puede observar en la Figura 12.

Nivel de gris	**Frecuencia**	**Probabilidad**	**CFD (I)**	**Ec. Exp**	**Ec.Exp*255**	**Iexp**
0	24	0.24	0.24	0.27	69.98	70
1	4	0.04	0.28	0.328	83.77	84
2	5	0.05	0.33	0.400	102.12	102
3	1	0.01	0.34	0.415	105.96	106
6	2	0.02	0.36	0.446	113.80	114
7	1	0.01	0.37	0.462	117.82	118
13	1	0.01	0.38	0.478	121.90	122
23	1	0.01	0.39	0.494	126.05	126
34	1	0.01	0.4	0.510	130.26	130
40	1	0.01	0.41	0.527	134.55	135
45	1	0.01	0.42	0.544	138.91	139
47	1	0.01	0.43	0.562	143.34	143
75	1	0.01	0.44	0.579	147.85	148
77	1	0.01	0.45	0.597	152.45	152
96	1	0.01	0.46	0.616	157.13	157
108	1	0.01	0.47	0.634	161.89	162
112	1	0.01	0.48	0.653	166.75	167
122	1	0.01	0.49	0.673	171.70	172
133	1	0.01	0.5	0.693	176.75	177
140	4	0.04	0.54	0.776	198.01	198
141	4	0.04	0.58	0.867	221.21	221
142	15	0.15	0.73	1.309	255,00	255
143	17	0.17	0.9	2.302	255.00	255
144	3	0.03	0.93	2.659	255.00	255
146	1	0.01	0.94	2.813	255.00	255
147	3	0.03	0.97	3.506	255.00	255
149	2	0.02	0.99	4.605	255.00	255
150	1	0.01	1	0.0001	0.0300	0

Tabla 1: Tabla completa con valores de ecualización exponencial.

255	255	221	255	255	198	255	139	70	102
255	255	255	255	198	255	162	84	84	70
255	255	255	255	198	147	143	70	102	70
255	255	255	255	255	172	114	70	70	70
255	255	255	221	255	152	70	102	70	70
255	255	255	221	255	135	70	102	70	70
255	255	255	255	177	122	70	84	70	70
255	255	255	255	167	70	70	70	70	114
255	255	221	255	157	70	102	70	84	126
255	255	198	0	148	70	106	70	118	130

Figura 12: Imagen ecualizada de la Figura 11

A continuación, se describe la función *ecualización_exponencial* que aplica la ecualización exponencial a una imagen digital. Los parámetros de entrada son los siguientes:

- La variable *imagen* es la imagen a procesar , representada como una matriz de números enteros o flotantes, que contiene los valores de intensidad de los píxeles.
- La variable *cfd* hace referencia a la función de distribución acumulada, es el arreglo que almacena las probabilidades acumuladas de cada nivel de intensidad calculadas a partir del histograma de la imagen.
- La variable *alpha* es un parámetro escalar definido como 1 en el código y controla la intensidad de la transformación exponencial aplicada a los niveles de intensidad de la imagen.

El parámetro de salida es *imagen_exponencial*, esta variable almacena el resultado de aplicar la transformación exponencial a la imagen de entrada *imagen*.

La función normaliza la imagen, calcula el histograma para obtener la CDF y aplica la transformación exponencial en la línea 14 del Listing 1. Finalmente, la imagen es escalada nuevamente a niveles de intensidad entre 0 y 255, lo que permite visualizar el resultado con mayor facilidad.

```
1  def ecualizacion_exponencial():
2      if img1 is None:
3          return messagebox.showerror("Error", "Debe seleccionar una imagen")
4      imagen = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
5      imagen_normalizada = imagen / 255.0
6      histograma, _ = np.histogram(imagen, bins=256, range=(0, 255))
7
8      cdf = np.cumsum(histograma) / np.sum(histograma)
9
10     alpha = 1
11     g_min = 0
12
13     cdf = np.clip(cdf, 0, 1 - 1e-6)
14     imagen_exponencial = g_min - (1 / alpha) * np.log(1 - cdf[imagen.astype(
           int)])
15     imagen_exponencial = np.clip(imagen_exponencial * 255, 0, 255).astype(np
           .uint8)
16     mostrar_imagen(imagen_exponencial)
```

Listing 1: Función de ecualización exponencial

4.4. Eliminación de ruido

El ruido en una imagen digital se refiere a la presencia de variaciones aleatorias e indeseadas en la intensidad de los píxeles, que afectan negativamente la calidad visual y la información contenida en la imagen. Estas perturbaciones pueden surgir durante la captura, procesamiento o transmisión de la imagen y pueden ser provocadas por factores como interferencias electrónicas, condiciones de iluminación inadecuadas, limitaciones del sensor de imagen o errores en la compresión de datos (González & Woods, 2018).

Existen distintos tipos de ruido, entre los más comunes se encuentra el ruido gaussiano y el rudio sal y pimienta, ejemplificados en la Figura 13; sin embargo existen diferentes técnicas para eliminarlos, por ejemplo, el filtro gaussiano y el filtro moda.

Figura 13: a) Imagen mujer rubia color con ruido gaussiano . b) Imagen mujer rubia color con ruido sal y pimienta

4.4.1. Ruido gaussiano

En este tipo de ruido los valores de los píxeles de la imagen se alteran de acuerdo con una distribución normal (o gaussiana); una imagen con ruido gaussiano suele tener un aspecto "granulado"debido a las pequeñas alteraciones en todos los píxeles.

Ahora se presenta un ejemplo de aplicación de ruido gaussiano:

Sea $I_(x, y)$ una imagen brillante con dimensión de 10x10 con un rango de valores entre 0 y 255 (Figura 14).

190	190	194	198	204	209	214	217	216	220
185	191	198	203	207	210	215	217	217	220
190	197	203	208	210	211	214	216	219	222
195	201	206	208	209	210	214	216	220	223
198	202	206	207	206	209	215	218	222	221
238	235	239	244	247	248	247	248	248	248
243	238	240	247	247	248	248	249	248	248
244	239	242	247	248	249	249	249	249	248
244	241	244	247	249	249	249	249	248	248
245	242	245	248	249	249	249	249	249	248

Figura 14: Imagen brillante

Los cálculos para la aplicación de ruido gaussiano en los primeros cinco píxeles de la Figura 14 se aprecia en la Tabla 2, considerando un ruido generado aleatoriamente que sigue una distribución normal con $\mu = 0$ y $\sigma = 5$. La imagen completa con ruido gaussiano se muestra en la Figura 15.

Píxel	**Valor Original**	**Ruido $\mathcal{N}(0,5)$**	**Valor Transformado**	**Valor Ajustado**
(0,0)	190	+2.7	192.7	193
(0,1)	190	-1.4	188.6	189
(0,2)	194	+3.6	197.6	198
(0,3)	198	-2.1	195.9	196
(0,)	204	+4.8	208.8	209

Tabla 2: Generación de ruido gaussiano y ajuste de valores de píxeles

193	188	198	194	205	207	217	218	215	221
182	194	196	205	206	211	216	215	219	218
190	196	205	209	206	213	211	216	222	220
194	204	204	210	207	206	216	216	221	223
199	202	207	209	204	213	213	218	222	220
237	239	236	247	247	249	247	248	248	248
244	240	240	247	247	248	250	249	247	248
245	240	245	249	249	248	250	248	247	249
245	243	244	249	251	248	249	249	248	249
247	240	246	249	248	249	249	252	248	249

Figura 15: Imagen con ruido gaussiano de la Figura 14

A continuación se describe la función *generar_ruido_gaussiano* para aplicar ruido gaussiano a una imagen digital. Los parámetros de entrada son:

- La variable *img1* es una matriz que representa la imagen original, convertida previamente a escala de grises si es necesario.
- La variable *mean* es la media, hace referencia al valor promedio de la distribución normal, que controla el nivel de brillo del ruido.
- La variable *std_dev* es la desviación estándar, valor que determina la dispersión del ruido, afectando su intensidad.

El parámetro de salida es *imagen_resultado*, hace referencia a la imagen transformada con ruido gaussiano aplicado a las posiciones calculadas a *img1*.

La función genera ruido gaussiano de forma aleatoria, añadiéndolo a la imagen original. Los valores del ruido se calculan siguiendo una distribución normal determinada por la media y la desviación estándar, especificadas en las líneas 11 y 12 del Listing 2. Posteriormente, este ruido es sumado a la imagen en escala de grises, que se obtiene como una copia de la original, asegurando que los valores de intensidad

resultantes estén dentro del rango válido mediante la operación realizada en la línea 15.

```
1  def generar_ruido_gaussiano():
2      global img_resultado
3      if img1 is None:
4          return messagebox.showerror("Error", "Debe seleccionar una imagen")
5
6      if len(img1.shape) == 3:
7          imagen_gris = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
8      else:
9          imagen_gris = img1.copy()
10
11     mean = 0
12     std_dev = 25
13     ruido = np.random.normal(mean, std_dev, imagen_gris.shape).astype(np.
           float32)
14
15     img_resultado = np.clip(imagen_gris + ruido, 0, 255).astype(np.uint8)
16
17     mostrar_imagen(cv2.cvtColor(img_resultado, cv2.COLOR_GRAY2BGR))
```

Listing 2: Código para generar ruido gaussiano en una imagen

4.4.2. Ruido sal y pimienta

En el ruido sal y pimienta los valores de los píxeles de la imagen se alteran de manera abrupta, reemplazándose por valores extremos, típicamente 0 (negro) o 255 (blanco) por lo que la imagen se ve "salpicada.º con pequeños puntos blancos y negros distribuidos aleatoriamente (González, 2007).

Enseguida se presenta un ejemplo de aplicación de ruido sal y pimienta: Sea $I_(x, y)$ una imagen de alto contraste con dimensión de 10x10 con un rango de valores entre 0 y 255 (Figura 16).

75	78	83	91	91	94	103	110	113	121
75	77	80	89	91	93	103	110	115	123
74	77	79	88	91	93	103	111	118	126
74	77	80	86	91	94	103	114	121	130
74	77	81	86	90	97	107	117	127	135
75	78	82	86	90	101	116	123	132	140
75	80	85	88	94	109	120	129	139	145
75	80	86	89	98	112	124	137	146	153
75	81	85	93	106	118	132	144	152	160
75	82	87	97	112	122	135	148	157	162

Figura 16: Imagen con alto contraste

El cálculo para la aplicación de ruido sal y pimienta en los primeros cinco píxeles se aprecia en la Tabla 3, tomando en cuenta que la probabilidad de que cada píxel sea modificado, se determina según las siguientes reglas:

- Si el número aleatorio generado está en el rango de 0 a 0,1 (10 %), el valor del píxel se reemplaza por pimienta (0).

- Si el número aleatorio está entre 0,1 y 0,2 (10 %), el valor se reemplaza por sal (255).
- Si el número aleatorio se encuentra en el intervalo de 0,2 a 1 (80 %), el píxel conserva su valor original.

Posteriormente la imagen con ruido sal y pimienta se puede apreciar en la Figura 17.

Posición	**Valor Original**	**Número Aleatorio**	**Probabilidad**	**Nuevo Valor**
(0,0)	75	0.05	0 - 0.1	0
(0,1)	78	0.15	0.1 - 0.2	255
(0,2)	83	0.35	0.2 - 1	83
(0,3)	91	0.8	0.2 - 1	91
(0,4)	91	0.09	0 - 0.1	0

Tabla 3: Cálculo de ruido sal y pimienta para los primeros 5 píxeles de la Figura 14

0	255	83	91	0	94	103	255	113	121
75	77	80	255	91	93	103	110	115	123
74	77	79	88	91	0	103	111	118	126
74	77	80	0	91	94	103	255	121	0
74	77	255	86	90	255	107	117	127	0
255	78	255	86	90	101	116	123	132	140
75	255	85	255	94	109	120	0	139	145
75	80	86	89	98	112	124	137	146	153
75	255	85	0	106	118	132	144	152	160
75	255	0	97	112	122	135	148	157	162

Figura 17: Imagen con ruido sal y pimienta de la Figura 16

Enseguida se describe la función *generar_ruido_sal_pimienta* que aplica ruido sal y pimienta a una imagen digital. Los parámetros de entrada son los siguientes:

- La variable *imagen_gris* es una matriz que representa la imagen original, convertida previamente a escala de grises si es necesario.
- La variable *prob_ruido* es la probabilidad de ruido, un valor flotante que define la proporción de píxeles en la imagen que serán afectados por el ruido.
- La variable *cantidad_salpimienta* es el número total de píxeles alterados, calculado a partir del tamaño de la imagen y la probabilidad de ruido.

El parámetro de salida es *imagen_resultado*, hace referencia a la imagen transformada, que incluye con ruido sal y pimienta aplicado a las posiciones calculadas a *imagen_gris*.

La función selecciona de manera aleatoria en la imagen las posiciones de los píxeles "sal y pimienta", basándose en las coordenadas generadas en las líneas 14 y 18 del Listing 3. Estos píxeles son modificados directamente sobre una copia de la imagen original.

En la línea 24 del Listing 3, se encuentra la instrucción principal para asignar píxeles "sal"(blancos), mientras que en la línea 25 para asignar píxeles "pimienta"(negros).

```
1  def generar_ruido_sal_pimienta():
2      global img_resultado
3      if img1 is None:
4          return messagebox.showerror("Error", "Debe seleccionar una imagen")
5
6      if len(img1.shape) == 3:
7          imagen_gris = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
8      else:
9          imagen_gris = img1.copy()
10
11     prob_ruido = 0.05
12     cantidad_salpimienta = int(prob_ruido * imagen_gris.size)
13
14     coords_sal = [
15         np.random.randint(0, i - 1, cantidad_salpimienta // 2) for i in
               imagen_gris.shape
16     ]
17
18     coords_pimienta = [
19         np.random.randint(0, i - 1, cantidad_salpimienta // 2) for i in
               imagen_gris.shape
20     ]
21
22     img_resultado = imagen_gris.copy()
23
24     img_resultado[coords_sal[0], coords_sal[1]] = 255
25     img_resultado[coords_pimienta[0], coords_pimienta[1]] = 0
26
27     mostrar_imagen(cv2.cvtColor(img_resultado, cv2.COLOR_GRAY2BGR))
```

Listing 3: Función de ruido sal y pimienta

4.4.3. Filtro gaussiano

Como señala González & Woods (2018), el filtro gaussiano está diseñado para reducir el ruido y suavizar la imagen de manera controlada; es ideal para preservar las transiciones suaves mientras elimina las variaciones rápidas, este filtro se basa en la convolución de la imagen con un núcleo que sigue una distribución Gaussiana, dicho núcleo está definido por la función:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

Donde:

- x, y**:** Coordenadas espaciales del punto en el plano bidimensional. x representa la distancia horizontal y y la distancia vertical desde el origen.
- σ**:** Desviación estándar de la distribución gaussiana.

  Un valor grande de σ genera una curva más ancha y difusa.
  Un valor pequeño de σ produce una curva más estrecha y concentrada.

Retomando la Figura 15, se presenta un ejemplo para aplicar filtro gaussiano a dicha figura. Sin embargo, antes de aplicar este filtro, es necesario agregar ceros alrededor de la imagen original (Figura 18). Este

relleno, conocido como padding, permite que el filtro se aplique correctamente en los bordes de la imagen. Al agregar columnas y filas con ceros, se asegura que el filtro tenga píxeles vecinos en todas las áreas, incluso en las esquinas, lo que garantiza que la convolución se realice de manera adecuada en toda la imagen.

0	0	0	0	0	0	0	0	0	0	0	0
0	193	188	198	194	205	207	217	218	215	221	0
0	182	194	196	205	206	211	216	215	219	218	0
0	190	196	205	209	206	213	211	216	222	220	0
0	194	204	204	210	207	206	216	216	221	223	0
0	199	202	207	209	204	213	213	218	222	220	0
0	237	239	236	247	247	249	247	248	248	248	0
0	244	240	240	247	247	248	250	249	247	248	0
0	245	240	245	249	249	248	250	248	247	249	0
0	245	243	244	249	251	248	249	249	248	249	0
0	247	240	246	249	248	249	249	252	248	249	0
0	0	0	0	0	0	0	0	0	0	0	0

Figura 18: Imagen con padding de la Figura 15

En la Tabla 4 se presentan los valores necesarios en la aplicación del filtro gaussiano a la imagen. En ella, se muestra el resultado de la convolución de algunos píxeles con su vecindad 3×3 correspondiente, el resultado de la convolución y el valor final obtenido para cada píxel tomando en cuenta que el valor de la distribución estándar es $\sigma = 1$. La aplicación del filtro gaussiano se muestra en la Figura 19.

Píxel	**Valor Original**	**Vecindad 3×3**	**Convolución**	**Valor Final**
(0,0)	193	0 0 0 0 193 188 0 182 194	1076	107
(1,1)	194	193 182 190 188 194 196 198 196 205	2966	185
(1,2)	196	182 190 194 194 196 204 196 205 209	2988	186
(1,3)	204	190 194 199 196 204 202 205 209 210	3010	188
(1,4)	202	194 199 237 204 202 239 209 210 247	3172	198
(1,5)	239	199 237 244 202 239 240 210 247 247	3338	208

Tabla 4: Resultado de la convolución con el filtro gaussiano.

107	184	189	209	196	239	243	244	244	243
186	185	186	188	198	208	240	240	243	240
188	190	188	202	204	236	240	245	244	246
198	198	198	198	199	247	247	249	249	249
199	200	200	200	202	247	247	249	251	248
199	202	204	200	204	249	248	248	248	249
204	205	204	204	205	247	250	250	249	249
205	204	204	204	205	248	249	248	249	252
205	204	205	205	205	248	247	247	248	248
208	208	208	208	208	248	248	249	249	249

Figura 19: Imagen con filtro gaussiano de la Figura 15

De igual manera se describe la función *aplicar_filtro_gaussiano* que aplica filtro gaussiano a una imagen digital. El parámetro de entrada es la imagen original (*img1)* mientras que el parámetro de salida *imagen_resultado*, hace referencia a la imagen transformada una vez aplicado el filtro.

En la línea 5 del Listing 4, se utiliza la función GaussianBlur de OpenCV, la cual tiene tres parámetros: el primero es la imagen original sobre la cual se aplicará el filtro, en este caso es la variable *img1*; el segundo hace referencia al tamaño del kernel y el tercero corresponde a la desviación estándar en el eje x.

```
1 def aplicar_filtro_gaussiano():
2     global img_resultado
3     if img1 is None:
4         return messagebox.showerror("Error", "Debe seleccionar una imagen")
5     img_resultado = cv2.GaussianBlur(img1, (5, 5), 0)
6     mostrar_imagen(img_resultado)
```

Listing 4: Función de Filtro Gaussiano

4.4.4. Filtro moda

Por otro lado, el filtro moda selecciona el mas frecuente (moda) de los valores en la vecindad de un píxel. Es muy efectivo para eliminar ruido sal y pimienta y gracias a su capacidad para preservar los bordes, este filtro no difumina los contornos, lo que ayuda a mantener detalles importantes en la imagen; es especialmente útil en aplicaciones que requieren la conservación de bordes y detalles finos mientras se elimina el ruido intenso (Castleman, 1996).

Ahora se presenta un ejemplo para aplicar filtro moda a una imagen.

Sea *I(x, y)* una imagen obscura con dimensión de 10x10 con un rango de valores entre 0 y 255 (Figura 20).

5	10	12	16	20	21	22	23	28	27
4	7	9	12	15	15	16	16	18	19
2	5	7	10	14	12	12	12	15	14
1	3	5	9	11	12	10	10	10	11
1	2	4	7	10	10	8	8	8	9
1	1	2	3	4	5	7	7	6	9
0	1	2	4	4	3	4	4	4	4
0	0	0	1	3	4	4	3	3	4
0	0	0	0	1	2	4	3	3	4
0	0	0	0	0	1	2	2	3	2

Figura 20: Imagen obscura

Como en el ejemplo anterior del filtro gaussiano, es indispensable agregar padding para asegurar que el filtro tenga píxeles vecinos en todas las áreas, esto puede observarse en la Figura 21.

0	0	0	0	0	0	0	0	0	0	0	0
0	5	10	12	16	20	21	22	23	28	27	0
0	4	7	9	12	15	15	16	16	18	19	0
0	2	5	7	10	14	12	12	12	15	14	0
0	1	3	5	9	11	12	10	10	10	11	0
0	1	2	4	7	10	10	8	8	8	9	0
0	1	1	2	3	4	5	7	7	6	9	0
0	0	1	2	4	4	3	4	4	4	4	0
0	0	0	0	1	3	4	4	3	3	4	0
0	0	0	0	0	1	2	4	3	3	4	0
0	0	0	0	0	0	1	2	2	3	2	0
0	0	0	0	0	0	0	0	0	0	0	0

Figura 21: Imagen con padding de la Figura 20

En la Tabla 5 se presentan los detalles obtenidos al aplicar el filtro moda a la imagen. La tabla incluye información clave de algunos píxeles, como sus coordenadas en la matriz original, el valor inicial del píxel, su vecindad 3x3, el valor o los valores moda encontrados en dicha vecindad y el nuevo valor asignado tras aplicar el filtro.

Para este ejemplo, se tomó como criterio seleccionar el valor menor entre los valores moda encontrados en la vecindad. Este enfoque permite preservar detalles más oscuros en la imagen. Sin embargo, el criterio de selección puede variar según el objetivo del análisis. Por ejemplo, si se busca resaltar áreas brillantes, podría elegirse el valor mayor. La imagen completa con filtro moda se muestra en la Figura 22.

Píxel	**Valor Original**	**Vecindad** 3×3	**Moda**	**Valor Final**
(0,0)	5	0 0 0 0 5 10 0 4 7	0	0
(1,0)	10	0 0 0 5 10 12 4 7 9	0	0
(1,1)	7	5 10 12 4 7 9 2 5 7	7,5	5
(2,1)	9	10 12 16 7 9 12 5 7 10	7,10,12	7
(3,1)	12	12 0 0 5 10 12 7 10 14	12	12

Tabla 5: Resultado de la convolución con el filtro gaussiano.

0	0	0	0	0	0	0	0	0	0
0	5	7	12	12	12	12	12	12	0
0	5	5	9	12	12	12	10	10	0
0	1	5	7	10	10	12	8	8	0
0	1	2	4	10	10	10	8	8	0
0	1	2	4	4	4	4	4	4	0
0	0	1	4	4	4	4	4	4	0
0	0	0	0	4	4	4	4	4	0
0	0	0	0	0	4	2	3	3	0
0	0	0	0	0	0	0	0	0	0

Figura 22: Imagen con filtro moda de la Figura 20

Ahora se muestra la función *aplicar_filtro_moda* para aplicar filtro moda a una imagen digital. Los parámetros de entrada son:

- La variable *img1* es una matriz que representa la imagen original.
- La variable *kernel_size* es el tamaño del kernel, hace referencia al valor que determina el tamaño del vecindario de píxeles que se toma en cuenta para aplicar el filtro.

El parámetro de salida es *img_resultado*, que corresponde a la imagen transformada después de aplicar el filtro de moda.

En la línea 17 del Listing 5, se utiliza la función mode de SciPy para calcular la moda de los valores dentro del vecindario de la imagen. Esta función toma como entrada un vector de valores (en este caso, el vecindario de píxeles del kernel de la imagen) y devuelve el valor que más veces se repite, es decir, la moda.

```
1  def aplicar_filtro_moda():
2      global img_resultado
3      if img1 is None:
4          return messagebox.showerror("Error", "Debe seleccionar una imagen")
5
6      kernel_size = 3
7      pad_size = kernel_size // 2
8
9      img_padded = np.pad(img1, ((pad_size, pad_size), (pad_size, pad_size),
           (0, 0)), mode='constant', constant_values=0)
10
11     img_resultado = np.zeros_like(img1)
12
13     for i in range(img1.shape[0]):
14         for j in range(img1.shape[1]):
15             for c in range(img1.shape[2]):  # Canales de color (BGR)
16                 vecindario = img_padded[i:i + kernel_size, j:j + kernel_size
                       , c]
17                 moda = mode(vecindario.flatten())[0][0]
18                 img_resultado[i, j, c] = moda
19
20     mostrar_imagen(img_resultado)
```

Listing 5: Función de Filtro Moda

4.5. Segmentación parcial y total

En la segmentación de imágenes existen dos enfoques principales: la segmentación parcial y la segmentación total.

La segmentación parcial se centra en aislar áreas específicas dentro de una imagen basándose en criterios como bordes, textura o color, sin procesar toda la imagen. Este enfoque es ideal para aplicaciones donde se desea estudiar zonas específicas de interés (González & Woods, 2018).En este contexto, la detección de bordes, mediante filtros como el Filtro Roberts, desempeña un papel crucial al identificar las fronteras donde se producen cambios bruscos en la intensidad de los píxeles, permitiendo resaltar y delimitar con mayor precisión dichas áreas de interés (Russ, 2016).

Por otro lado, la segmentación total tiene como objetivo dividir toda la imagen en segmentos significativos, asignando etiquetas a cada píxel que corresponden a diferentes objetos o estructuras dentro de la escena.

4.5.1. Filtro Roberts

El filtro de Roberts (también conocido como operador de Roberts) es un método para la detección de bordes en imágenes digitales, calcula el gradiente de intensidad en direcciones diagonales, dicho gradiente representa cambios abruptos en la intensidad, que suelen corresponder a bordes o límites entre diferentes objetos o regiones dentro de la imagen, es relavante mencionar que este operador no suaviza la imagen antes de calcular el gradiente, por lo que es más adecuado para imágenes limpias (Pratt, 2007).

El filtro utiliza dos máscaras 2x2 para calcular las derivadas parciales en las direcciones diagonales principales. Estas máscaras son:

$$G_x = \begin{bmatrix} +1 & 0 \\ 0 & -1 \end{bmatrix}, \quad G_y = \begin{bmatrix} 0 & -1 \\ +1 & 0 \end{bmatrix}$$

Donde:
G_x = Máscara que detecta bordes en la dirección horizontal.
G_y = Máscara que detecta bordes en la dirección vertical.

La magnitud del gradiente, que representa la fuerza del borde, se calcula combinando G_x y G_y. Matemáticamente:

$$M = \sqrt{G_x^2 + G_y^2}$$

En implementaciones prácticas, la fórmula anterior normalmente se aproxima para reducir el costo computacional:

$$M \approx |G_x| + |G_y|$$

A continuación se presenta un ejemplo del filtro roberts aplicado a la figura 11.

En la Tabla 6 se presentan los detalles obtenidos al aplicar el filtro roberts a la imagen. La tabla incluye información clave de algunos píxeles, como sus coordenadas en la matriz original, el valor inicial del píxel, su vecindad 2x2, los valores de las máscaras G_x yG_y, el valor de la magnitud y el nuevo valor asignado después de aplicar el filtro.

Píxel	**Valor Original**	**Vecindad** 2×2	**(Gx, Gy)**	**Magnitud (M)**	**Valor Final**
(0,0)	142	142 142 143 143	(-1,-1)	$\sqrt{2}$	1.4
(1,0)	142	142 141 143 142	(0,-2)	$\sqrt{4}$	2
(1,1)	143	143 142 142 142	(1,0)	$\sqrt{1}$	1
(2,1)	142	142 142 142 142	(0,0)	0	0
(3,1)	142	142 140 142 140	(2,-2)	$\sqrt{8}$	2.8

Tabla 6: Resultado de la convolución con el filtro gaussiano.

1.4	2	1	2	4	32	156.2	44	1
1	1	0	2.8	9.2	106.4	117.4	1.4	2.24
0	1.4	1.4	3.6	18.2	159.7	47.4	2	2
0	1	2	6.71	72.2	141.1	4	2	0
1	1	2.83	10	129.5	86.7	2.8	2.8	0
1	1	1	8.9	163.1	42	2.2	2.2	0
1.4	1.4	5.1	34	165.8	13	1	1	6
1	2.2	9.2	63	147.5	2	2	1	23.5
1	3.61	12.7	91.61	121.8	3.6	3.6	7.1	36.6

Figura 23: Imagen con filtro roberts de la Figura 11

A continuación se describe la función *filtro_roberts*, que aplica el filtro de Roberts a una imagen digital para la segmentación parcial. El parámetro de entrada es la imagen original (*img1)* mientras que el parámetro de salida *imagen_resultado*, hace referencia a la imagen transformada una vez aplicado el filtro.

En las líneas 15 y 16 del Listing 6, se utiliza la función filter2D de OpenCV para aplicar el filtro de Roberts, el cual calcula las componentes del gradiente de la imagen. Esta función tiene tres parámetros: el primero es la imagen en escala de grises sobre la cual se realiza la operación, en este caso la variable *imagen_gris*; el segundo es el tipo de profundidad de los datos de la imagen resultante, definido como *CV_64F* para permitir valores flotantes; y el tercero es el kernel de convolución utilizado para calcular la derivada.

En la línea 15, el kernel empleado es *kernel_gx*, que genera la componente horizontal del gradiente almacenada en la variable *gx*. De manera similar, en la línea 16 se utiliza *kernel_gy* como kernel de convolución para calcular la componente vertical del gradiente, almacenada en la variable *gy*.

```
1    def filtro_roberts():
2        global img_resultado
3        if img1 is None:
4            return messagebox.showerror("Error", "Debe seleccionar una imagen")
5
6        if len(img1.shape) == 3:
7            imagen_gris = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
8        else:
9            imagen_gris = img1.copy()
10
11       kernel_gx = np.array([[1, 0], [0, -1]], dtype=np.float32)
12       kernel_gy = np.array([[0, 1], [-1, 0]], dtype=np.float32)
13
14
15       gx = cv2.filter2D(imagen_gris, cv2.CV_64F, kernel_gx)  # Usar CV_64F para
             valores flotantes
16       gy = cv2.filter2D(imagen_gris, cv2.CV_64F, kernel_gy)
17
18       magnitud = np.sqrt(gx**2 + gy**2)
19       img_resultado = cv2.convertScaleAbs(magnitud)  # Escalar al rango de 0 a 255
20
21       mostrar_imagen(cv2.cvtColor(img_resultado, cv2.COLOR_GRAY2BGR))
```

Listing 6: Función de Filtro Robert

4.5.2. Umbral banda

El umbral banda es una técnica utilizada en PDI para segmentar o resaltar píxeles que se encuentran dentro de un rango específico de valores de intensidad. En lugar de establecer un único valor umbral como en el umbral simple, el umbral banda define un intervalo mediante dos límites: un valor mínimo y un valor máximo. Solo los píxeles con valores de intensidad dentro de este rango serán seleccionados o modificados, mientras que los demás serán descartados o asignados a un valor predeterminado (Chandran, 2010).

Para un píxel con valor de intensidad $I(x, y)$, los umbrales inferiores y superiores se definen como $T_{\min}$ y $T_{\max}$, respectivamente. La fórmula es:

$$I'(x, y) = \begin{cases} I(x, y) & \text{si } T_{\min} \leq I(x, y) \leq T_{\max} \\ 0 & \text{en otro caso} \end{cases}$$

Donde:

- $I(x, y)$ = Valor de intensidad del píxel original en la posición (x, y).
- $I'(x, y)$ = Valor de intensidad del píxel procesado en la posición (x, y).
- $T_{\min}$ y $T_{\max}$ = Límites inferior y superior de la banda de interés.

El valor del píxel permanece inalterado si su intensidad cae dentro del rango $[T_{\min}, T_{\max}]$.

Enseguida se muestra la función *umbral_ banda* que aplica el método de umbralización por banda a una imagen digital. El parámetro de entrada es *img1* correspondiente a la imagen original mientras que el parámetro de salida *img_ resultado* hace referencia a la imagen procesada después de aplicar el umbral.

En las líneas 17 y 18 del Listing 7, se utiliza la función cv2.threshold de OpenCV para calcular los umbrales inferior y superior de la imagen esta función tiene cuatro parámetros principales:

- La variable *imagen_ gris* es la imagen en escala de grises sobre la cual se realiza la operación.
- La variable *t1* o *t2* es el valor del umbral, que define el límite inferior o superior correspondientemente.
- El valor escalar 255 hace referencia al valor máximo que se asignará a los píxeles cuya intensidad en el caso de la línea 17 sea mayor o igual al umbral *t1*, y mayor o igual al umbral *t2* en el caso de la línea 18.
- El parámetro *cv2.THRESH_ BINARY* en la linea 17 es una constante predefinida en OpenCV para la umbralización bnaria.
- El parámetro *cv2.THRESH_ BINARY_ INV* es una constante utilizada en OpenCV para aplicar un umbral binario invertido.

En la línea 19 del Listing 7, se utiliza la operación *cv2.bitwise_ and(umbral_ inferior, umbral_ superior)* para realizar una operación lógica AND entre las dos imágenes binarizadas *umbral_ inferior* y *umbral_ superior*.

Esta operación devuelve una nueva imagen en la que solo los píxeles que son blancos (255) en ambas imágenes de entrada permanecen blancos en la imagen resultante. Los demás píxeles se convierten en negros (0), aislando así las áreas de interés que cumplen con los umbrales definidos.

```python
def umbral_banda():
    global img_resultado
    if img1 is None:
        return messagebox.showerror("Error", "Debe seleccionar una imagen")

    # Convertir a escala de grises si es necesario
    if len(img1.shape) == 3:
        imagen_gris = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
    else:
        imagen_gris = img1.copy()

    # Definir los valores umbral
    t1 = int(entry_t1.get())  # Umbral inferior
    t2 = int(entry_t2.get())  # Umbral superior

    # Aplicar Umbral Banda
    _, umbral_inferior = cv2.threshold(imagen_gris, t1, 255, cv2.THRESH_BINARY)
    _, umbral_superior = cv2.threshold(imagen_gris, t2, 255, cv2.
        THRESH_BINARY_INV)
    img_resultado = cv2.bitwise_and(umbral_inferior, umbral_superior)

    # Mostrar la imagen resultante
    mostrar_imagen(cv2.cvtColor(img_resultado, cv2.COLOR_GRAY2BGR))
```

Listing 7: Función de Umbral Banda

5. Resultados

A continuación se presentan los resultados obtenidos con base a las pruebas y experimentación:

5.1. Pruebas en imágenes obscuras

La Figura 24 muestra las etapas aplicadas para lograr la segmentación total del rostro de la imagen 4-a. En la imagen 24-b se realiza un aumento de brillo utilizando un alpha de 0.9 y un beta de 50, siendo el parámetro beta el que incrementa la luminosidad global sumando un valor constante a cada píxel, lo cual permite mejorar la visibilidad de los detalles iniciales.

Posteriormente, en la imagen 24-c, se lleva a cabo la conversión a escala de grises. Este paso es esencial, ya que reduce la información de la imagen a un solo canal de intensidad, lo que facilita la aplicación de técnicas de detección de bordes. A partir de esta imagen en escala de grises, en la imagen 24-d se realiza la segmentación parcial, donde se aplican los filtros Roberts y Canny para destacar variaciones abruptas de intensidad, detectando contornos iniciales y refinandolos.

En la siguiente etapa, mostrada en la imagen 24-e, se implementa la segmentación total mediante el método de Otsu y binarización. Esta técnica es especialmente útil para imágenes con bajo contraste, como es el caso de la imagen inicial, ya que permite separar de manera óptima el objeto principal del fondo. El resultado es una imagen binaria clara, en la que el rostro queda segmentado completamente.

Finalmente, en la imagen f, se realiza una operación AND entre la imagen original y la binarizada, con el objetivo de validar los bordes detectados. Esta operación permite superponer los contornos segmentados sobre la imagen original, asegurando que los bordes verificando que correspondan correctamente a las características del rostro.

a. b. c. d. e. f.

Figura 24: a) Imagen mujer obscura. b) Imagen con ajuste de brillo y contraste. c) Imagen ajustada en escala de grises d) Segmentación parcial. e) Segmentación total f) Validación de bordes con imagen original

5.2. Pruebas en imágenes brillantes

En la Figura 25 se muestran las fases aplicadas para obtener la segmentación total del rostro presente en la imagen 5-a.

En la imagen 25-b se utiliza un alpha de 0.8 ya que un valor de alpha menor que 1 siempre atenuará el brillo para reducir la intensidad del brillo de la imagen. Además se aplica corrección gamma, calculando el valor de gamma más adecuado para corregir el contraste y adaptar la imagen a un rango de brillo específico, optimizando dichocontraste y asegurando un ajuste eficiente mediante la determinación del valor de gamma ideal. Se aplica un filtro Gaussiano con un kernel de tamaño (5, 5). Este filtro suaviza la imagen, reduciendo así el contraste al difuminar los detalles y suavizando las transiciones de color.

Posteriormente, en la imagen 25-c se lleva a cabo la conversión a escala de grises; se realiza la segmentación parcial con ayuda de los filtros Roberts y Canny obteniendo la imagen 25-d. La segmentación total

mostrada en la imagen 25-e se obuvo por medio del Método Otsu y de la binarización. Para validar que la segmentación total fue la correcta, se aplica una operación AND entre la imagen original y la binarizada verificando que correspondan correctamente a las características del rostro, dicha operación se puede apreciar en la imagen 25-f.

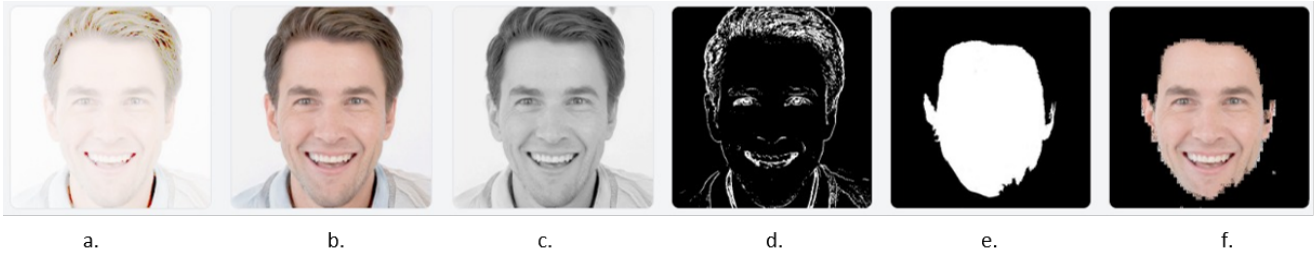


Figura 25: a) Imagen hombre brillante. b) Imagen con ajuse de brillo y contraste. c) Imagen ajustada en escala de grises d) Segmentación parcial. e) Segmentación total f) Validación de bordes con imagen original

5.3. Pruebas en imágenes con alto contraste

En la Figura 26 se muestran las fases aplicadas para obtener la segmentación total del rostro presente en la imagen 6-a. Para este tipo de imágenes, en las que las diferencias entre las áreas claras y oscuras son demasiado marcadas, se utilizó un filtro de suavizado para reducir la intensidad de esas transiciones. Se aplicó un filtro Gaussiano con un kernel de tamaño (5, 5) para suavizar la imagen, disminuyendo las transiciones abruptas de color y suavizando los detalles. Esto ayudó a crear una imagen más equilibrada, sin los fuertes bordes que caracterizan al alto contraste.

Posteriormente, se lleva a cabo la conversión a escala de grises (imagen 26-c); se realiza la segmentación parcial, con ayuda de los filtros Roberts y Canny (imagen 26-d); la segmentación total presentada en la imagen 26-e se obuvo por medio del Método Otsu y de la binarización. Para validar que la segmentación total fue la correcta, se aplica una operación AND entre la imagen original y la binarizada verificando que correspondan correctamente a las características del rostro, esto puede observarse en la imagen 26-f.

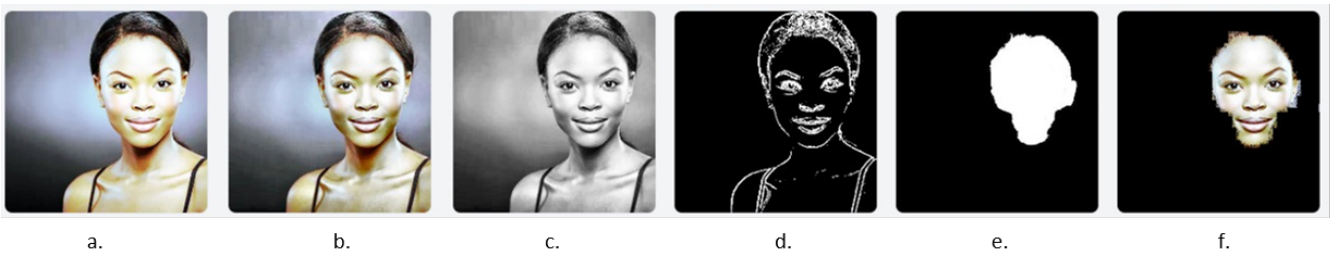


Figura 26: a) Imagen mujer alto contraste. b) Imagen con ajuste de brillo y contraste. c) Imagen ajustada en escala de grises d) Segmentación parcial. e) Segmentación total f) Validación de bordes con imagen original

5.4. Pruebas en imágenes con bajo contraste

En la Figura 27 se muestran las fases aplicadas para obtener la segmentación total del rostro presente en la imagen 7-a. Para este tipo de imágenes, en donde las diferencias entre las áreas claras y oscuras eran muy pequeñas, se aplicó la técnica de corrección gamma para mejorar la distinción entre los tonos. Se utilizó un valor de gamma de 0.3, lo que permitió aumentar significativamente el brillo en las áreas oscuras. Este

ajuste mejoró la visibilidad de los detalles, pero también redujo ligeramente el contraste en las áreas de transición, ya que las diferencias sutiles de color se hicieron menos evidentes (imagen 27-b).

Posteriormente se puede apreciar la conversión a escala de grises en la imagen 27-c; se realiza la segmentación parcial, con ayuda de los filtros Roberts y Canny (imagen 27-d); la segmentación total se obuvo por medio del Método Otsu y de la binarización (imagen 27-e). Para validar que la segmentación total fue la correcta se presenta la imagen 27-f, en la cual se aplica una operación AND entre la imagen original y la binarizada verificando que correspondan correctamente a las características del rostro.

Figura 27: a) Imagen mujer bajo contraste. b) Imagen con ajuse de brillo y contraste. c) Imagen ajustada en escala de grises d) Segmentación parcial. e) Segmentación total f) Validación de bordes con imagen original

En este capítulo se demostró que las técnicas implementadas fueron efectivas para mejorar la calidad de diferentes tipos de imágenes y facilitar la segmentación facial. La ecualización exponencial resultó útil para optimizar el brillo en imágenes obscuras y brillantes, mejorando la visibilidad de los detalles.

El filtro gaussiano fue efectivo para suavizar transiciones abruptas en imágenes con alto contraste, mientras que el filtro moda mostró un excelente desempeño al eliminar ruido sal y pimienta sin comprometer los bordes. Las técnicas de segmentación parcial mediante los filtros Roberts y Canny se aplicaron con éxito a todos los tipos de imágenes, detectando bordes relevantes. Finalmente, el método de Otsu y la binarización permitieron realizar una segmentación total efectiva, independientemente de las características iniciales de las imágenes.

6. Conclusiones

- En este trabajo se realizó una investigación exhaustiva sobre técnicas de ajuste de brillo, eliminación de ruido y detección de bordes.
- Para el ajuste de brillo, se implementó la técnica de “Ecualización Exponencial”, probándola en imágenes con diferentes características, como baja y alta luminosidad, así como contrastes extremos.
- Para la eliminación de ruido, se programaron y aplicaron los “Filtro Gaussiano” y “ Filtro Moda” en imágenes afectadas por ruido gaussiano y sal y pimienta, evaluando su efectividad en estos escenarios.
- Para detección de bordes se implementaron las técnicas “Filtro Roberts” y “Umbral banda” para delimitar la región de interés.
- Se integraron todos los prototipos para obtener la segmentación parcial y total de rostros.

Cada una de las tareas realizadas permitió que el objetivo general se cumpliera, por lo tanto, de acuerdo con las pruebas y experimentos se logró observar que de forma general no se pueden resolver estos problemas.

Anexos

Anexo 1. Aplicación para experimentar las técnicas de PDI

```
"""
Nombre: Aplicaci n para experimentar las t cnicas de PDI
Unidad de aprendizaje: Procesamiento Digital de Imagenes
Grupo 4BM1-Comunidad 3

Descripcion:
    Aplicaci n dise ada para experimentar con diversas t cnicas
    de Procesamiento Digital de Im genes (PDI), facilitando el
    an lisis y la manipulaci n de im genes mediante ajustes de
    brillo, eliminaci n de ruido y detecci n de bordes,

Tecnologias:
    -OpenCV
    -NumPy
    -Tkinter
    -Pillow
    -Matplotlib
    -SciPy
"""

import cv2
import numpy as np
import tkinter as tk
from tkinter import ttk
from tkinter import messagebox
from tkinter import filedialog
from PIL import Image, ImageTk
import matplotlib.pyplot as plt
from scipy.stats import mode

# ============================
# Cargar imagenes
# ============================

def mostrar_imagen1(img):
    """
    -Muestra la primera imagen en la interfaz grafica.
    -Args:
        img (numpy.ndarray): Imagen en formato NumPy que se desea mostrar.
    -Returns:
        None
    """

    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    img_pil = Image.fromarray(img_rgb)
    img_tk = ImageTk.PhotoImage(image=img_pil)
    btn_imagen1.config(image=img_tk)
    btn_imagen1.image = img_tk


def mostrar_imagen2(img):
    """
    -Muestra la segunda imagen en la interfaz grafica.
    -Args:
        img (numpy.ndarray): Imagen en formato NumPy que se desea mostrar.
    -Returns:
```

```
        None
    """

    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    img_pil = Image.fromarray(img_rgb)
    img_tk = ImageTk.PhotoImage(image=img_pil)
    btn_imagen2.config(image=img_tk)
    btn_imagen2.image = img_tk


def mostrar_imagen(img):
    """
    -Muestra la imagen resultante en la interfaz grafica.
    -Args:
        img (numpy.ndarray): Imagen en formato NumPy que se desea mostrar.
    -Returns:
        None
    """

    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    img_pil = Image.fromarray(img_rgb)
    img_tk = ImageTk.PhotoImage(image=img_pil)
    btn_imagen_resultado.config(image=img_tk)
    btn_imagen_resultado.image = img_tk


def cargar_imagen1():
    """
    -Carga la primera imagen desde el sistema de archivos y la muestra en la interfaz.
    -Args:
        None
    -Returns:
        None
    """

    filepath = filedialog.askopenfilename(filetypes=[("Image files", "*.jpg;*.png;*.jpeg
        ;*.bmp")])
    if filepath:
        global img1
        img1 = cv2.imread(filepath)
        img1 = cv2.resize(img1, (150, 150))
        mostrar_imagen1(img1)


def cargar_imagen2():
    """
    -Carga la segunda imagen desde el sistema de archivos y la muestra en la interfaz.
    -Args:
        None
    -Returns:
        None
    """

    filepath = filedialog.askopenfilename(filetypes=[("Image files", "*.jpg;*.png;*.jpeg
        ;*.bmp")])
    if filepath:
        global img2
        img2 = cv2.imread(filepath)
        img2 = cv2.resize(img2, (150, 150))
        mostrar_imagen2(img2)

```

```python

# ============================
# Conversi n de im genes
# ============================

def convertir_escala_grises():
    """
    -Convierte la imagen cargada a escala de grises y la muestra en la interfaz.
    -Args:
        None
    -Returns:
        None
    """

    global img_resultado
    if img1 is None:
        return messagebox.showerror("Error", "Debe seleccionar una imagen")

    if img_resultado is None:
        img_resultado = img1.copy()

    gris = cv2.cvtColor(img_resultado, cv2.COLOR_BGR2GRAY)
    mostrar_imagen(cv2.cvtColor(gris, cv2.COLOR_GRAY2BGR))


# ================================
# Operaciones aritmeticas y logicas
# ================================

def operacion_aritmetica(operacion):
    """
    -Realiza una operacion aritmetica entre dos imagenes cargadas.
    -Args:
        operacion (str): Operacion a realizar ('Suma', 'Resta', 'Multiplicacion').
    -Returns:
        None
    """

    global img_resultado
    if img1 is None or img2 is None:
        return messagebox.showerror("Error", "Debe seleccionar dos im genes antes de
            realizar alguna operaci n")

    if img1.shape != img2.shape:
        return messagebox.showerror("Error", "Las im genes deben tener las mismas
            dimensiones")

    if operacion == 'Suma':
        img_resultado = cv2.add(img1, img2)
    elif operacion == 'Resta':
        img_resultado = cv2.subtract(img1, img2)
    elif operacion == 'Multiplicaci n':
        img_resultado = cv2.multiply(img1, img2)
    else:
        return messagebox.showerror("Error", "Operaci n no v lida")

    mostrar_imagen(img_resultado)


def operacion_aritmetica_escalar(operacion):
    """
```

```
    -Realiza una operacion aritmetica entre una imagen y un escalar.
    -Args:
        operacion (str): Operacion a realizar ('Desplazamiento', 'Contraccion', 'Expansion').
    -Returns:
        None
    """

    global img_resultado
    if img1 is None:
        return messagebox.showerror("Error", "Debe seleccionar una imagen antes de realizar la operaci n")

    if img_resultado is None:
        img_resultado = img1.copy()

    valor_escalar = valida_escalar()
    if valor_escalar is None:
        return

    if operacion == 'Desplazamiento':
        img_resultado = cv2.add(img_resultado, valor_escalar)
    elif operacion == 'Contracci n':
        img_resultado = cv2.multiply(img_resultado, valor_escalar)
    elif operacion == 'Expansi n':
        img_resultado = cv2.subtract(img_resultado, valor_escalar)
    else:
        return messagebox.showerror("Error", "Operaci n no v lida")

    mostrar_imagen(img_resultado)

def operacion_logica(operacion):
    """
    -Realiza una operaci n logica entre dos imagenes cargadas.
    -Args:
        operacion (str): Operacion logica a realizar ('AND', 'OR', 'XOR').
    -Returns:
        None
    """

    global img_resultado
    if img1 is None or img2 is None:
        return messagebox.showerror("Error", "Debe seleccionar dos im genes antes de realizar alguna operaci n")

    if operacion == 'AND':
        img_resultado = cv2.bitwise_and(img1, img2)
    elif operacion == 'OR':
        img_resultado = cv2.bitwise_or(img1, img2)
    elif operacion == 'XOR':
        img_resultado = cv2.bitwise_xor(img1, img2)
    else:
        return messagebox.showerror("Error", "Operaci n no v lida")

    mostrar_imagen(img_resultado)

# ============================
# Ruido y eliminacion de ruido
# ============================

def generar_ruido_gaussiano():
```

```
    """
    -A ade ruido gaussiano a la imagen cargada.
    -Args:
        None
    -Returns:
        None
    """
    global img_resultado
    if img1 is None:
        return messagebox.showerror("Error", "Debe seleccionar una imagen")

    if img_resultado is None:
        img_resultado = img1.copy()

    mean = 0
    std_dev = 25
    ruido = np.random.normal(mean, std_dev, img_resultado.shape).astype(np.float32)
    img_resultado = np.clip(img_resultado + ruido, 0, 255).astype(np.uint8)
    mostrar_imagen(img_resultado)


def generar_ruido_sal_pimienta():
    """
    -A ade ruido de tipo sal y pimienta a la imagen cargada.
    -Args:
        None
    -Returns:
        None
    """

    global img_resultado
    if img1 is None:
        return messagebox.showerror("Error", "Debe seleccionar una imagen")

    if img_resultado is None:
        img_resultado = img1.copy()

    prob_ruido = 0.05
    cantidad_salpimienta = int(prob_ruido * img_resultado.size)
    coords_sal = [np.random.randint(0, i - 1, cantidad_salpimienta // 2) for i in
        img_resultado.shape]
    coords_pimienta = [np.random.randint(0, i - 1, cantidad_salpimienta // 2) for i in
        img_resultado.shape]

    img_resultado[coords_sal[0], coords_sal[1], :] = 255
    img_resultado[coords_pimienta[0], coords_pimienta[1], :] = 0
    mostrar_imagen(img_resultado)


# ============================
# Deteccion de bordes
# ============================

def deteccion_bordes():
    """
    -Aplica el detector de bordes de Canny a la imagen cargada.
    -Args:
        None
    -Returns:
        None
    """
```

```python

    global img_resultado
    if img1 is None:
        return messagebox.showerror("Error", "Debe seleccionar una imagen")

    if img_resultado is None:
        img_resultado = img1.copy()

    gris = cv2.cvtColor(img_resultado, cv2.COLOR_BGR2GRAY)
    img_resultado = cv2.Canny(gris, 100, 200)
    mostrar_imagen(cv2.cvtColor(img_resultado, cv2.COLOR_GRAY2BGR))


def filtro_roberts():
    """
    -Aplica el filtro de deteccion de bordes de Roberts a la imagen cargada.
    -Args:
        None
    -Returns:
        None
    """

    global img_resultado
    if img1 is None:
        return messagebox.showerror("Error", "Debe seleccionar una imagen")

    if img_resultado is None:
        img_resultado = img1.copy()

    imagen_gris = cv2.cvtColor(img_resultado, cv2.COLOR_BGR2GRAY)

    kernel_gx = np.array([[1, 0], [0, -1]], dtype=np.float32)
    kernel_gy = np.array([[0, 1], [-1, 0]], dtype=np.float32)

    gx = cv2.filter2D(imagen_gris, cv2.CV_64F, kernel_gx)
    gy = cv2.filter2D(imagen_gris, cv2.CV_64F, kernel_gy)

    magnitud = np.sqrt(gx**2 + gy**2)
    img_resultado = cv2.convertScaleAbs(magnitud)
    mostrar_imagen(cv2.cvtColor(img_resultado, cv2.COLOR_GRAY2BGR))


# ============================
# Interfaz grafica
# ============================

# Crear la ventana principal
root = tk.Tk()
root.title("Procesamiento de Im genes Avanzado")
root.geometry("1200x800")

# Crear marco para cargar imagenes y botones
frame_cargar = tk.Frame(root, padx=10, pady=10)
frame_cargar.grid(row=0, column=0, columnspan=3, pady=10, sticky="nsew")

# Crear marco para mostrar imagenes
frame_imagenes = tk.Frame(root, padx=10, pady=10, relief=tk.SUNKEN, bd=2)
frame_imagenes.grid(row=1, column=0, columnspan=3, pady=10, sticky="nsew")

# Crear marco para operaciones
frame_operaciones = tk.Frame(root, padx=10, pady=10)
```

```
frame_operaciones.grid(row=3, column=0, columnspan=3, pady=10, sticky="nsew")

# Botones para cargar imagenes
tk.Button(frame_cargar, text="Cargar imagen 1", command=cargar_imagen1, width=15).grid(row=0, column=0, padx=10)
tk.Button(frame_cargar, text="Cargar imagen 2", command=cargar_imagen2, width=15).grid(row=0, column=1, padx=10)

# Etiquetas para mostrar las imagenes
lbl_imagen1 = tk.Label(frame_imagenes, relief=tk.RIDGE)
lbl_imagen1.grid(row=0, column=0, padx=10, pady=10)

lbl_imagen2 = tk.Label(frame_imagenes, relief=tk.RIDGE)
lbl_imagen2.grid(row=0, column=1, padx=10, pady=10)

lbl_imagen_resultado = tk.Label(frame_imagenes, relief=tk.RIDGE)
lbl_imagen_resultado.grid(row=0, column=2, padx=10, pady=10)

# Crear un marco principal para las secciones
frame_principal = tk.Frame(root, padx=10, pady=10)
frame_principal.grid(row=3, column=0, columnspan=3, sticky="nsew")

# Configurar columnas del marco principal
frame_principal.grid_columnconfigure(0, weight=1)
frame_principal.grid_columnconfigure(1, weight=1)
frame_principal.grid_columnconfigure(2, weight=1)
frame_principal.grid_columnconfigure(3, weight=1)
frame_principal.grid_columnconfigure(4, weight=1)

# Crear botones para las operaciones
# Operaciones Aritmeticas
frame_oper_arit = tk.Frame(frame_principal, padx=10, pady=10, relief=tk.GROOVE, bd=2)
frame_oper_arit.grid(row=0, column=0, padx=10, pady=10, sticky="nsew")
tk.Label(frame_oper_arit, text="Operaciones Aritm ticas", font=("Arial", 10, "bold")).pack(pady=5)

operaciones_aritmeticas = ttk.Combobox(frame_oper_arit, values=["Suma", "Resta", "Multiplicaci n"], width=20)
operaciones_aritmeticas.pack(pady=5)
btn_aplicar_aritmetica = tk.Button(frame_oper_arit, text="Aplicar", command=lambda: operacion_aritmetica(operaciones_aritmeticas.get()))
btn_aplicar_aritmetica.pack(pady=5)

# Operaciones Escalares
frame_oper_escalar = tk.Frame(frame_principal, padx=10, pady=10, relief=tk.GROOVE, bd=2)
frame_oper_escalar.grid(row=0, column=1, padx=10, pady=10, sticky="nsew")
tk.Label(frame_oper_escalar, text="Operaciones Escalares", font=("Arial", 10, "bold")).pack(pady=5)

entry_escalar = tk.Entry(frame_oper_escalar, width=15)
entry_escalar.pack(pady=5)
tk.Button(frame_oper_escalar, text="Desplazamiento", command=lambda: operacion_aritmetica_escalar('Desplazamiento'), width=20).pack(pady=5)
tk.Button(frame_oper_escalar, text="Contracci n", command=lambda: operacion_aritmetica_escalar('Contracci n'), width=20).pack(pady=5)
tk.Button(frame_oper_escalar, text="Expansi n", command=lambda: operacion_aritmetica_escalar('Expansi n'), width=20).pack(pady=5)

# Operaciones Logicas
frame_oper_logica = tk.Frame(frame_principal, padx=10, pady=10, relief=tk.GROOVE, bd=2)
frame_oper_logica.grid(row=0, column=2, padx=10, pady=10, sticky="nsew")
```

```
tk.Label(frame_oper_logica, text="Operaciones L gicas", font=("Arial", 10, "bold")).pack(pady=5)

operaciones_logicas = ttk.Combobox(frame_oper_logica, values=["AND", "OR", "XOR"], width=20)
operaciones_logicas.pack(pady=5)
btn_operacion_logica = tk.Button(frame_oper_logica, text="Aplicar", command=lambda: operacion_logica(operaciones_logicas.get()))
btn_operacion_logica.pack(pady=5)

# Filtros de Ruido
frame_filtros = tk.Frame(frame_principal, padx=10, pady=10, relief=tk.GROOVE, bd=2)
frame_filtros.grid(row=0, column=3, padx=10, pady=10, sticky="nsew")
tk.Label(frame_filtros, text="Ruido y Filtros", font=("Arial", 10, "bold")).pack(pady=5)

btn_ruido_gaussiano = tk.Button(frame_filtros, text="Ruido Gaussiano", command=generar_ruido_gaussiano, width=20)
btn_ruido_gaussiano.pack(pady=5)

btn_ruido_sal_pimienta = tk.Button(frame_filtros, text="Ruido Sal y Pimienta", command=generar_ruido_sal_pimienta, width=20)
btn_ruido_sal_pimienta.pack(pady=5)

# Deteccion de Bordes
frame_bordes = tk.Frame(frame_principal, padx=10, pady=10, relief=tk.GROOVE, bd=2)
frame_bordes.grid(row=0, column=4, padx=10, pady=10, sticky="nsew")
tk.Label(frame_bordes, text="Detecci n de Bordes", font=("Arial", 10, "bold")).pack(pady=5)

btn_canny = tk.Button(frame_bordes, text="Bordes Canny", command=deteccion_bordes, width=20)
btn_canny.pack(pady=5)

btn_roberts = tk.Button(frame_bordes, text="Filtro Roberts", command=filtro_roberts, width=20)
btn_roberts.pack(pady=5)

# Boton para salir
btn_salir = tk.Button(root, text="Salir", command=root.quit, width=20, bg="red", fg="white")
btn_salir.grid(row=4, column=0, columnspan=3, pady=20)

# Iniciar el loop de la interfaz
tk.mainloop()
```

Anexo 2.Aplicación para la segmentación parcial y total de rostros

```python

"""
Nombre de la aplicacion: FaceTrack
Unidad de aprendizaje: Procesamiento Digital de Imagenes
Grupo 4BM1-Comunidad 3

Descripcion:
    Aplicacion para procesar y analizar imagenes subidas por el usuario
    utilizando tecnicas de ajuste de brillo, eliminacion de ruido y deteccion
    de bordes para lograr segmentar parcial y totalmente un rostro.

Tecnologias:
    #-Flask
    #-OpenCV
    #-NumPy
    #-PIL
"""


from flask import Flask, request, jsonify, send_from_directory
from flask_cors import CORS
from PIL import Image
import numpy as np
import cv2
import os


# ============================================
# Configuracion de Flask y carpetas de trabajo
# ============================================

app = Flask(__name__)
CORS(app)

UPLOAD_FOLDER = 'uploads'
PROCESSED_FOLDER = 'processed'
MASK_FOLDER = 'mask'
os.makedirs(UPLOAD_FOLDER, exist_ok=True)
os.makedirs(PROCESSED_FOLDER, exist_ok=True)
os.makedirs(MASK_FOLDER, exist_ok=True)

image_paths = {
    'Imagen1_obscura.bmp': 'Imagen1_obscura_mask.jpeg',
    'Imagen2_brillante.bmp': 'Imagen2_brillante_mask.jpeg',
    'Imagen3_alto contraste.bmp': 'Imagen3_alto contraste_mask.jpeg',
    'Imagen4_bajo contraste.bmp': 'Imagen4_bajo contraste_mask.jpeg'
}


# ============================================
# Carga y preprocesamiento de imagenes
# ============================================

def load_and_preprocess_image(filepath):
    """
    -Carga una imagen, analiza su brillo y contraste, y realiza ajustes necesarios.
    -Args:
        filepath (str): Ruta de la imagen a cargar.
    -Returns:
```

```
        tuple: Imagen procesada (grayscale), rutas de la imagen original,
               ajustada y en escala de grises.
    """

    image = cv2.imread(filepath)
    if image is None:
        raise ValueError('No se pudo cargar la imagen')

    original_path = os.path.join(PROCESSED_FOLDER, "original_" + os.path.basename(filepath))
    cv2.imwrite(original_path, image)

    brightness_level, contrast_level = analyze_image_properties(image)
    res_image = adjust_image(image, brightness_level, contrast_level)
    adjusted_path = os.path.join(PROCESSED_FOLDER, "adjusted_" + os.path.basename(filepath))
    cv2.imwrite(adjusted_path, res_image)

    res_image = cv2.cvtColor(res_image, cv2.COLOR_BGR2GRAY)
    grayscale_path = os.path.join(PROCESSED_FOLDER, "grayscale_" + os.path.basename(filepath))
    cv2.imwrite(grayscale_path, res_image)

    return res_image, original_path, adjusted_path, grayscale_path


# ============================================
# Ajuste de brillo y contraste
# ============================================

def analyze_image_properties(image):
    """
    -Analiza las propiedades de brillo y contraste de una imagen.
    -Args:
        image (np.ndarray): Imagen a analizar.
    -Returns:
        tuple: Niveles de brillo y contraste clasificados.
    """
    brightness = np.mean(image)
    contrast = np.std(image)

    if brightness > 200:
        brightness_level = "brillante"
    elif brightness < 50:
        brightness_level = "oscura"
    else:
        brightness_level = "normal"

    if contrast > 70:
        contrast_level = "alto_contraste"
    elif contrast < 30:
        contrast_level = "bajo_contraste"
    else:
        contrast_level = "contraste_normal"

    return brightness_level, contrast_level


def adjust_image(image, brightness_level, contrast_level):
    """
    -Ajusta la imagen seg n los niveles de brillo y contraste detectados.
```

```
    -Args:
        -image (np.ndarray): Imagen a ajustar.
        -brightness_level (str): Nivel de brillo detectado.
        -contrast_level (str): Nivel de contraste detectado.
    -Returns:
        np.ndarray: Imagen ajustada.
    """

    if brightness_level == "brillante" and contrast_level == "alto_contraste":
        result_image = cv2.convertScaleAbs(image, alpha=0.8, beta=-30)
        result_image = cv2.GaussianBlur(result_image, (5, 5), 0)
    elif brightness_level == "brillante" and contrast_level == "bajo_contraste":
        gamma = calculate_gamma(image)
        result_image = adjust_gamma(image, gamma)
    elif brightness_level == "oscura" and contrast_level == "alto_contraste":
        result_image = cv2.convertScaleAbs(image, alpha=0.9, beta=50)
    elif brightness_level == "oscura" and contrast_level == "bajo_contraste":
        result_image = adjust_gamma(image, 0.3)
    elif brightness_level == "normal" and contrast_level == "alto_contraste":
        result_image = soft_image(image)
    elif brightness_level == "normal" and contrast_level == "bajo_contraste":
        result_image = cv2.equalizeHist(image)
    else:
        result_image = cv2.normalize(image, None, 0, 255, cv2.NORM_MINMAX)
    return result_image


def calculate_gamma(image):
    """
    -Calcula el valor gamma basado en el brillo promedio de la imagen.
    -Args:
        image (np.ndarray): Imagen a analizar.
    -Returns:
        float: Valor de gamma calculado.
    """

    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    brightness_mean = np.mean(gray)
    ideal_brightness = 128
    gamma = 1.0 + (brightness_mean - ideal_brightness) / 32 if brightness_mean >
        ideal_brightness else 1.0 - (ideal_brightness - brightness_mean) / 32
    return max(0.1, min(gamma, 5.0))


def adjust_gamma(image, gamma):
    """
    -Aplica correccion gamma a la imagen.
    -Args:
        -image (np.ndarray): Imagen a ajustar.
        -gamma (float): Valor gamma para la correcci n.
    -Returns:
        np.ndarray: Imagen con gamma ajustado.
    """

    lookUpTable = np.empty((1, 256), np.uint8)
    for i in range(256):
        lookUpTable[0, i] = np.clip(pow(i / 255.0, gamma) * 255.0, 0, 255)
    return cv2.LUT(image, lookUpTable)


def soft_image(image):
```

```
    """
    -Suaviza la imagen utilizando el modelo de color LAB y mejora el contraste.
    -Args:
        image (np.ndarray): Imagen a suavizar.
    -Returns:
        np.ndarray: Imagen suavizada y mejorada.
    """

    lab_image = cv2.cvtColor(image, cv2.COLOR_BGR2LAB)
    l, a, b = cv2.split(lab_image)
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
    l_clahe = clahe.apply(l)
    lab_clahe = cv2.merge((l_clahe, a, b))
    result_image = cv2.cvtColor(lab_clahe, cv2.COLOR_LAB2BGR)
    gamma = calculate_gamma(result_image)
    return adjust_gamma(result_image, gamma)


# ============================================
# Funciones para ruido y filtrado
# ============================================

def denoise_image(image):
    """
    -Aplica un filtro Gaussiano para reducir el ruido en la imagen.
    -Args:
        image (np.ndarray): Imagen con ruido.
    -Returns:
        np.ndarray: Imagen con ruido reducido.
    """

    return cv2.GaussianBlur(image, (5, 5), 0)


def enhance_contrast(image):
    """
    -Mejora el contraste de una imagen utilizando ecualizacion de histograma.
    -Args:
        image (np.ndarray): Imagen a mejorar.
    -Returns:
        np.ndarray: Imagen con contraste mejorado.
    """
    return cv2.equalizeHist(image)


# ============================================
# Funciones de segmentacion y deteccion de bordes
# ============================================

def filtro_roberts(image):
    """
    -Aplica el filtro de deteccion de bordes de Roberts.
    -Args:
        image (np.ndarray): Imagen de entrada.
    -Returns:
        np.ndarray: Imagen con bordes detectados.
    """

    if len(image.shape) == 3:
        image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    kernel_gx = np.array([[1, 0], [0, -1]], dtype=np.float32)
```

```
    kernel_gy = np.array([[0, 1], [-1, 0]], dtype=np.float32)
    gx = cv2.filter2D(image, cv2.CV_64F, kernel_gx)
    gy = cv2.filter2D(image, cv2.CV_64F, kernel_gy)
    magnitud = np.sqrt(gx**2 + gy**2)
    magnitud = (magnitud / np.max(magnitud)) * 255 if np.max(magnitud) != 0 else
        magnitud
    return cv2.convertScaleAbs(magnitud)


def filtro_sobel(image):
    """
    -Aplica el filtro de deteccion de bordes de Sobel.
    -Args:
        image (np.ndarray): Imagen de entrada.
    -Returns:
        np.ndarray: Imagen con bordes detectados.
    """

    if len(image.shape) == 3:
        image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    sobel_x = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=3)
    sobel_y = cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=3)
    gradiente_magnitud = cv2.magnitude(sobel_x, sobel_y)
    return cv2.normalize(gradiente_magnitud, None, 0, 255, cv2.NORM_MINMAX)


def combinar_filtros(image1, weight_roberts, image2, weight_sobel, offset=0):
    """
    -Combina los resultados de los filtros de Roberts y Sobel.
    -Args:
        -image1 (np.ndarray): Resultado del filtro Roberts.
        -weight_roberts (float): Peso asignado al filtro Roberts.
        -image2 (np.ndarray): Resultado del filtro Sobel.
        -weight_sobel (float): Peso asignado al filtro Sobel.
        -offset (int): Valor de desplazamiento para ajustar el brillo.
    -Returns:
        np.ndarray: Imagen combinada con bordes detectados.
    """

    image1 = image1.astype(np.float32)
    image2 = image2.astype(np.float32)
    combinada = (weight_roberts * image1 + weight_sobel * image2 + offset)
    return np.clip(combinada, 0, 255).astype(np.uint8)


def partial_segmentation(image, filepath):
    """
    -Realiza una segmentacion parcial utilizando los filtros de Roberts y Sobel.
    -Args:
        -image (np.ndarray): Imagen a segmentar.
        -filepath (str): Ruta del archivo de imagen.
    -Returns:
        tuple: Imagen combinada y ruta de la imagen segmentada.
    """

    edges_roberts = filtro_roberts(image)
    edges_sobel = filtro_sobel(image)
    combined_edges = combinar_filtros(edges_roberts, 0.5, edges_sobel, 0.5)
    combined_edges = cv2.normalize(combined_edges, None, 0, 255, cv2.NORM_MINMAX)
    _, thresholded_image = cv2.threshold(combined_edges, 0, 255, cv2.THRESH_BINARY + cv2
        .THRESH_OTSU)
```

```
    thresholded_path = os.path.join(PROCESSED_FOLDER, "thresholded_" + os.path.basename(filepath))
    cv2.imwrite(thresholded_path, thresholded_image)
    return combined_edges, thresholded_path


# ============================================
# Interfaz Flask
# ============================================

@app.route('/processed/<path:filename>')
def serve_processed_file(filename):
    """
    -Sirve una imagen procesada desde la carpeta de archivos procesados.
    -Args:
        filename (str): Nombre del archivo.
    -Returns:
        Response: Archivo procesado como respuesta HTTP.
    """

    response = send_from_directory(PROCESSED_FOLDER, filename)
    response.headers['Cache-Control'] = 'no-store, no-cache, must-revalidate, max-age=0'
    response.headers['Pragma'] = 'no-cache'
    response.headers['Expires'] = '0'
    return response

@app.route('/upload', methods=['POST'])
def upload_file():
    """
    -Maneja la subida de archivos y procesa la imagen subida.
    -Returns:
        Response: JSON con las rutas de las imagenes procesadas o mensaje de error.
    """

    if 'image' not in request.files:
        return jsonify({'error': 'No se encontr  ninguna imagen'}), 400

    file = request.files['image']
    filename = os.path.basename(file.filename)
    filepath = os.path.join(UPLOAD_FOLDER, filename)
    file.save(filepath)

    try:
        preprocessed_image, original_path, adjusted_path, grayscale_path = load_and_preprocess_image(filepath)
        secondary_path, segmentation_path = partial_segmentation(preprocessed_image, filepath)

        return jsonify({
            "message": "Imagen procesada exitosamente",
            "original_image": original_path,
            "adjusted_image": adjusted_path,
            "grayscale_image": grayscale_path,
            "segmented_image": segmentation_path
        })
    except Exception as e:
        return jsonify({'error': str(e)}), 500

if __name__ == "__main__":
    app.run(port=3000, debug=True)
```

Referencias

- Castleman, K. R. (1996). *Digital Image Processing*. Prentice Hall.
- Gonzalez, R. C., & Woods, R. E. (2018). *Digital image processing* (4th ed.). Pearson.
- Gonzalez, R. (2007). *Digital Image Processing using MATLAB*. Pearson Prentice Hall.
- Jain, A. K. (1989). *Fundamentals of Digital Image Processing*. Prentice Hall.
- Pájarez Martinsanz, G., & de la Cruz García, J. M. (2023). *Procesamiento de imágenes digitales*. Editorial Académica.
- Pratt, W. K. (2007). *Digital Image Processing: PIKS Inside* (4th ed.). Wiley-Interscience.
- Russ, J. C. (2016). *The Image Processing Handbook* (7th ed.). CRC Press.
- Szeliski, R. (2010). *Computer Vision: Algorithms and Applications*. Springer.
- Zhao, W., Chellappa, R., & Rosenfeld, A. (2003). Face recognition: A literature survey. *ACM Computing Surveys (CSUR)*, 35(4), 399-458. https://doi.org/10.1145/954339.954342